

StarForth User Guide

Running, Scripting, and Exploring StarForth

Robert A. James

Contents

I	First Steps	1
1	Installation	3
1.1	Installation	3
1.1.1	Prerequisites	3
1.1.2	Building from Source	3
1.1.3	Installation Methods	4
1.1.4	Platform-Specific Notes	4
1.1.5	Verifying the Installation	5
1.1.6	Uninstallation	5
1.1.7	Troubleshooting	5
1.1.8	Building the Documentation	6
2	Quick Start	7
2.1	Quick Start	7
2.1.1	Common Build Commands	7
2.1.2	Platform-Specific Builds	7
2.1.3	Performance Modes	8
2.1.4	INIT System	8
2.1.5	Testing and Benchmarking	8
2.1.6	Flags Enabled by <code>make fastest</code>	8
2.1.7	Advanced Build Options	8
2.1.8	Troubleshooting	9
3	Your First Session	11
3.1	Starting StarForth	11
3.2	Pushing Numbers	11
3.3	Arithmetic	11
3.4	Inspecting the Stack with <code>.S</code>	12
3.5	Listing Available Words with <code>WORDS</code>	12
3.6	Defining Your First Word	12
3.7	Errors and Recovery	12
3.8	Exiting	13
3.9	Command-Line Shortcuts	13
II	The Stack	15
4	Stack Fundamentals	17
4.1	What the Stack Is	17
4.2	Stack Depth	17
4.3	The Data Stack vs. the Return Stack	17

5	Core Stack Words	19
5.1	Single-Item Operations	19
5.2	Double-Item Operations	20
5.3	The Return Stack	20
III	Arithmetic and Logic	23
6	Integer Arithmetic	25
6.1	Basic Arithmetic Words	25
6.2	Comparison Words	26
7	Logical and Bitwise Operations	27
7.1	Bitwise Words	27
8	Q48.16 Fixed-Point Arithmetic	29
8.1	What Q48.16 Is	29
8.2	When You See Q48.16	29
8.3	Q48.16 Words	29
IV	Defining Words	31
9	Colon Definitions	33
9.1	: and ;	33
9.2	A Worked Example	33
9.3	The SMUDGE Flag	34
9.4	IMMEDIATE	34
10	Variables, Constants, and CREATE	35
10.1	VARIABLE	35
10.2	CONSTANT	35
10.3	CREATE and DOES>	35
V	Control Flow	37
11	Conditionals and Loops	39
11.1	IF ... ELSE ... THEN	39
11.2	DO ... LOOP	39
11.3	BEGIN ... UNTIL	40
11.4	BEGIN ... WHILE ... REPEAT	40
11.5	A Note on Interactive Compilation	40
VI	Memory and Blocks	41
12	Memory Access	43
12.1	Cell-Sized Access: @ and !	43
12.2	Byte-Sized Access: C@ and C!	43
12.3	Bulk Memory: MOVE and FILL	43

13 The Block Subsystem	45
13.1 What Blocks Are	45
13.2 The VM Memory Map	45
13.3 Block Words	45
14 String and I/O Words	47
14.1 Output Words	47
14.2 Input Words	47
14.3 String Literals: S"	48
15 Running Files and Blocks	49
15.1 INCLUDE	49
15.2 EXEC	49
15.3 LOAD and THRU	49
16 The Interactive REPL	51
16.1 The ok> Prompt	51
16.2 Printing Numbers	51
16.3 Numeric Base	51
16.4 Dictionary Introspection	51
VII Physics Features	53
17 The Adaptive Runtime	55
17.1 What the Adaptive Runtime Does for You	55
17.2 The Seven Feedback Loops	55
17.3 The Seven Feedback Loops	55
17.3.1 Loop #1 — Execution Heat Tracking	55
17.3.2 Loop #2 — Rolling Window History	56
17.3.3 Loop #3 — Linear Decay	56
17.3.4 Loop #4 — Pipelining Metrics	56
17.3.5 Loop #5 — Window Width Inference	56
17.3.6 Loop #6 — Decay Slope Inference	57
17.3.7 Loop #7 — Adaptive Heartrate	57
17.3.8 Principal Stabilizers	57
17.3.9 Configuration	57
17.4 WORD-ENTROPY	58
17.5 PHYSICS-FREEZE and PHYSICS-THAW	58
17.6 PHYSICS-RESET-STATS	58
18 Heartbeat and Adaptive Tuning	59
18.1 Heartbeat CSV Export	59
18.1.1 CSV Format	59
18.1.2 Capturing Metrics	59
18.1.3 Analysis Use Cases	60
18.1.4 Implementation	60
18.1.5 Known Limitations	61

19 Design of Experiments Mode	63
19.1 The -doe Flag	63
19.2 Capturing Heartbeat Metrics	63
19.3 The 2 ⁷ Factorial Design	63
19.4 The Built-In Profiler	63
19.5 The Built-In Word Profiler	63
19.5.1 Quick Start	63
19.5.2 Profiling Levels	64
19.5.3 Reading the Report	64
19.5.4 Hot-Word Analysis	64
19.5.5 Development Workflow	64
19.5.6 Implementation	65
19.5.7 C API	65
19.5.8 Integration with External Tools	65
 VIII The ACL System	 67
20 Access Control for Interactive Users	69
20.1 What ACL Means for You	69
20.2 The ok> Prompt vs. Emergency ok>	69
20.3 The Zuse Session	69
20.4 Enabling the ACL System	70
 IX FORTH Programming	 71
21 Thinking in FORTH	73
21.1 Postfix Notation	73
21.2 Words Are Verbs	73
21.3 Factor Early, Factor Often	73
21.4 Stack Comments Are Documentation	74
21.5 The Interactive Development Loop	74
 22 Common Programming Patterns	 77
22.1 Accumulator Pattern	77
22.2 Table Dispatch	77
22.3 Counted String Pattern	78
22.4 Guard Clauses	78
22.5 State Machine in FORTH	78
 23 Debugging at the Prompt	 79
23.1 Using .S Liberally	79
23.2 SEE: Decompiling a Word	79
23.3 WORDS: Finding What Exists	79
23.4 Common Mistakes	80
 24 Working with Numbers	 81
24.1 Integer Ranges	81
24.2 Numeric Literals	81
24.3 Double-Cell Numbers	81
24.4 Pictured Numeric Output	82

25	Vocabularies and the Dictionary	83
25.1	How the Dictionary Is Organized	83
25.2	VOCABULARY	83
25.3	SEARCH-WORDLIST	84
26	Working with Text and I/O	85
26.1	Reading User Input	85
26.2	Parsing Input	85
26.3	Number Conversion	85
26.4	Formatted Output	86
27	Writing and Loading Source Files	87
27.1	The .4th File Convention	87
27.2	Comments in Source Files	87
27.3	Conditional Compilation	88
27.4	Loading Capsule Files	88
28	The Test Suite	89
28.1	Running the Tests	89
28.2	Writing Your Own Tests	89
X	Advanced Topics	91
29	The StarForth VM Internals for Users	93
29.1	How the Interpreter Loop Works	93
29.2	Direct Threading	93
29.3	The Hot-Words Cache	94
29.4	Execution Tokens	94
30	Performance Tuning Your Code	95
30.1	Measuring Before Optimizing	95
30.2	Build Profiles and Their Effect	95
30.3	Physics-Aware Programming	95
30.4	Stack Manipulation Cost	96
31	The Block Editor	97
31.1	Blocks as Source Storage	97
31.2	Block Storage Backends	97
31.3	Protecting Blocks with FLUSH	97
32	Capsules and the Boot System	99
32.1	What a Capsule Is	99
32.2	init.4th: The Default Personality	99
32.3	Capsule Variants	99
32.4	The Birth Protocol	100
33	The L8 Jacquard Mode Selector	101
33.1	What the Jacquard Selector Does	101
33.2	The Six Modes	101
33.3	Mode Selection for Users	101

34 Formal Verification	103
34.1 What Has Been Proved	103
34.2 What Determinism Means Here	103
34.3 Checking the Proofs	103
35 StarForth on L4Re and Embedded Targets	105
35.1 L4Re Microkernel	105
35.2 Minimal / Freestanding Builds	105
35.3 Cross-Compilation	105
XI Recipes	107
36 Frequently Used Idioms	109
36.1 Swap and Drop a Pair	109
36.2 Apply an Operation to Two Copies	109
36.3 Conditional Increment	110
36.4 Clamp a Value to a Range	110
36.5 Absolute Difference	110
36.6 Integer Division with Rounding	110
36.7 Swap Top Two Booleans	111
36.8 String Equality Test	111
36.9 Fibonacci Sequence	111
36.10 Printing a Separator Line	112
37 Working Examples	113
37.1 A Simple Stack Calculator	113
37.2 A Data Table	113
37.3 A Simple Moving Average	114
37.4 A Minimal State Logger	115
37.5 Physics-Aware Benchmarking	115
38 Quick Reference: Key Sequences	117
39 Migrating from Other FORTH Systems	119
39.1 From gforth or ANS FORTH	119
39.2 From FORTH-79	119
39.3 From C or Python	120
XII Running on Hardware	121
40 Linux and POSIX Platforms	123
41 Raspberry Pi 4	125
41.1 Building StarForth on Raspberry Pi 4	125
41.1.1 Building	125
41.1.2 Architecture Detection and Code Integration	125
41.1.3 Performance Tuning	125
41.1.4 Benchmarking	126
41.1.5 Thermal and Power	126
41.1.6 Storage and Deployment	126
41.1.7 Troubleshooting	126

42 LithosAnanke Bare-Metal Boot	127
42.1 Boot Sequence	127
42.2 Building the Kernel	127
A Build Flags Reference	129
A.1 Build Options Reference	129
A.1.1 Performance Optimization Options	129
A.1.2 Architecture Options	130
A.1.3 Platform / Target Options	130
A.1.4 Debugging and Profiling Options	131
A.1.5 Block System Options	131
A.1.6 Memory Configuration Options	131
A.1.7 Feature Flags	132
A.1.8 Build System Variables	132
A.1.9 Build Configuration Matrix	132
A.1.10 Optimization Level Guidelines	132
A.1.11 Usage Examples	133
A.1.12 Compiler Support	133
A.1.13 Performance Tuning Checklist	134
A.1.14 Troubleshooting	134
B FORTH-79 Word Quick Reference	135
C Glossary	143

About This Guide

This guide covers StarForth from the perspective of someone sitting at a terminal running it. You might know FORTH already, or this might be your first encounter with stack-based programming. Either way, the goal is the same: get you productive at the prompt.

StarForth is a FORTH-79 compliant virtual machine that also happens to contain a physics-driven adaptive runtime. Most of the time the runtime is invisible — it tunes itself in the background while you type. When you want to inspect or control it, this guide covers that too.

The guide is organized so you can read it front-to-back or jump to the chapter you need. Part I gets you running. Part II through Part VI cover the language. Part VII covers the physics features unique to StarForth. Part VIII gives a brief orientation to the access-control system. The appendix is a compact word-reference table you can keep open alongside your terminal.

Part I

First Steps

Chapter 1

Installation

1.1 Installation

StarForth builds from source on Linux (x86_64, ARM64) and experimentally on macOS and Windows (WSL2). No binary packages are distributed at this time.

1.1.1 Prerequisites

Required

- GCC $\geq$ 7.0 or Clang $\geq$ 6.0
- GNU Make
- glibc (standard builds)

Optional

- texinfo — GNU info documentation (`sudo apt-get install texinfo`)
- dpkg-dev — Debian package generation (`sudo apt-get install dpkg-dev`)
- rpm-build — RPM package generation (`sudo dnf install rpm-build`)

1.1.2 Building from Source

The standard optimized build:

```
1 make fastest
```

This produces the maximum-performance binary with ASM optimisations, direct threading, and LTO enabled. Table 1.1 lists all available build targets.

Target	Description	Performance tier
<code>make fastest</code>	Maximum performance build	Tier 5 (highest)
<code>make pgo</code>	Profile-guided optimisation	Tier 6 (5–15% above fastest)
<code>make all</code>	Standard optimised build	Tier 4
<code>make debug</code>	Debug build with symbols	Tier 1
<code>make minimal</code>	Minimal/embedded build	Tier 3

Table 1.1: StarForth build targets.

1.1.3 Installation Methods

System-Wide Installation (Recommended)

```
1 make fastest
2 sudo make install
```

Installs to the following paths:

- Binary: /usr/local/bin/starforth
- Config: /usr/local/etc/starforth/init.4th
- Man page: /usr/local/share/man/man1/starforth.1
- Documentation: /usr/local/share/doc/starforth/
- Info: /usr/local/share/info/starforth.info

Custom Prefix

```
1 make fastest
2 make install PREFIX=$HOME/.local
```

Debian Package

```
1 make deb
2 sudo dpkg -i ../starforth_1.1.0-1_amd64.deb
```

RPM Package

```
1 make rpm
2 sudo rpm -ivh ~/rpmbuild/RPMS/x86_64/starforth-1.1.0-1.x86_64.rpm
```

1.1.4 Platform-Specific Notes

x86_64 Linux

Standard installation works without modification:

```
1 make fastest
2 sudo make install
```

ARM64 (Raspberry Pi 4 and compatible)

Native build on ARM64:

```
1 make rpi4
2 sudo make install
```

Cross-compile from x86_64:

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make rpi4-cross
3 scp build/starforth user@arm64-device:~/
```

macOS (Experimental)

```

1 xcode-select --install
2 make fastest CC=clang
3 sudo make install

```

Windows (WSL2)

```

1 # Inside WSL2 (Ubuntu):
2 make fastest
3 make install PREFIX=$HOME/.local
4 echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
5 source ~/.bashrc

```

1.1.5 Verifying the Installation

```

1 starforth --version
2 starforth --run-tests
3 starforth

```

Quick functional check:

```

1 starforth -c ':_HELLO_"_Hello, _World!"_CR;_HELLO_BYE'
2 starforth --benchmark 10000

```

1.1.6 Uninstallation

```

1 # From source install:
2 sudo make uninstall
3
4 # From Debian package:
5 sudo dpkg -r starforth
6
7 # From RPM package:
8 sudo rpm -e starforth

```

1.1.7 Troubleshooting

gcc: command not found

```

1 # Debian/Ubuntu:
2 sudo apt-get install build-essential
3
4 # Fedora/RHEL:
5 sudo dnf groupinstall "Development Tools"

```

starforth: command not found after installation

```

1 which starforth
2 export PATH=$HOME/.local/bin:$PATH

```

Cannot find init.4th

```

1 export STARFORTH_INIT=/usr/local/etc/starforth/init.4th

```

1.1.8 Building the Documentation

Generate the PDF manual (LaTeX $\rightarrow$ PDF):

```
1 make book
```

Generate HTML documentation (single-page and multi-page):

```
1 make book-html
```

Chapter 2

Quick Start

2.1 Quick Start

Build the fastest binary for the current platform:

```
1 make fastest
```

2.1.1 Common Build Commands

Command	What it does	When to use
<code>make fastest</code>	Maximum speed build	Production, benchmarking
<code>make fast</code>	Fast without LTO	Development, debugging
<code>make debug</code>	Debug build (-g -O0)	Debugging with GDB
<code>make benchmark</code>	Run full benchmark suite	Performance testing
<code>make help</code>	List all options	Reference

Table 2.1: Common StarForth build commands.

2.1.2 Platform-Specific Builds

x86_64 Linux

```
1 make fastest          # Auto-detects x86_64; builds with ASM
   optimisations
2 ./build/starforth
```

Raspberry Pi 4 (native)

```
1 make fastest          # Auto-detects ARM64; optimised for Cortex
   -A72
2 ./build/starforth
```

Raspberry Pi 4 (cross-compile from x86_64)

Requires gcc-aarch64-linux-gnu:

```
1 make rpi4-cross        # Builds ARM64 binary with inline ASM
2 scp build/starforth pi@raspberrypi.local:~/
```

The resulting binary is statically linked and requires no runtime dependencies on the target device.

2.1.3 Performance Modes

Build targets are ranked by output speed:

1. **make pgo** — Profile-Guided Optimisation; 5–15% faster than **fastest**; requires two build passes (3–5 minutes).
2. **make fastest** — Recommended for production; ASM optimisations, direct threading, LTO.
3. **make fast** — ASM + direct threading; no LTO; easier to instrument.
4. **make turbo** — ASM optimisations only; no direct threading.
5. **make all** — Standard build (-O2).
6. **make debug** — No optimisations (-O0); full debug symbols.

2.1.4 INIT System

At startup StarForth automatically loads `./conf/init.4th`, which defines the foundational word set. No configuration is required:

```
1 ./build/starforth
2 ok>          # Words from init.4th are immediately available
```

`init.4th` defines words, installs a dictionary fence to protect them from `FORGET`, and zeros blocks for user use.

2.1.5 Testing and Benchmarking

```
1 make bench          # Quick benchmark (1 million operations)
2 make benchmark      # Full benchmark suite
3 make test           # Run full test suite (936+ tests)
```

Typical performance for 1 million stack operations:

Build type	x86_64	ARM64 (Raspberry Pi 4)
Debug	~800 ms	~1200 ms
Standard	~250 ms	~380 ms
Fastest	~60 ms	~95 ms
PGO	~50 ms	~80 ms

Table 2.2: StarForth benchmark timings (1M stack operations).

2.1.6 Flags Enabled by **make fastest**

2.1.7 Advanced Build Options

Custom compiler flags:

```
1 make fastest CFLAGS="$(make -s print-cflags)_-DCUSTOM_FLAG"
```

Flag	Effect
-O3	Maximum compiler optimisation
-march=native (x86_64)	Use all host CPU features
-march=armv8-a+crc+simd (ARM64)	ARMv8 with NEON
-DUSE_ASM_OPT=1	Assembly optimisations
-DUSE_DIRECT_THREADING=1	Direct-threaded interpreter
-flto	Link-Time Optimisation
-funroll-loops	Loop unrolling
-finline-functions	Aggressive function inlining
-fomit-frame-pointer	Free register for execution

Table 2.3: Compiler flags active in `make fastest`.

Static binary:

```
1 make fastest LDFLAGS="-static -s"
```

Alternate compiler:

```
1 make fastest CC=clang
```

2.1.8 Troubleshooting

Illegal instruction error The host CPU does not support a required optimisation. Use a conservative baseline:

```
1 make fastest CFLAGS="$ (BASE_CFLAGS) -O3 -march=x86-64 -DUSE_ASM_OPT=1"
```

Build fails outright Verify the standard build first, then re-enable optimisations incrementally:

```
1 make clean && make all
2 make clean && make fast
3 make clean && make fastest
```

Slow performance Confirm optimisation symbols are present in the binary:

```
1 file build/starforth
2 nm build/starforth | grep vm_push_asm
```


Chapter 3

Your First Session

This chapter walks through a complete interactive session from starting the REPL to shutting it down. Every prompt and response is annotated.

3.1 Starting StarForth

After building, start the interactive REPL:

```
1 ./build/amd64/standard/starforth
```

StarForth loads `init.4th`, sets up the dictionary, and prints the prompt:

```
1 ok>
```

The `ok>` prompt means the system is ready, the data stack is empty, and you are in interpretation mode. Everything you type is interpreted immediately. There is no compile-then-run cycle for interactive work.

3.2 Pushing Numbers

Type a number and press Enter. StarForth pushes it onto the data stack:

```
1 ok> 42
2 ok>
```

No output yet — the number is on the stack waiting to be used. Push another:

```
1 ok> 7
2 ok>
```

Now two numbers are on the stack: 42 on the bottom, 7 on top.

3.3 Arithmetic

The word `+` pops the top two items, adds them, and pushes the result:

```
1 ok> 42 7 +
2 ok>
```

Still no output — the result (49) is sitting on the stack. Use the dot word `.` to pop and print it:

```
1 ok> 42 7 + .
2 49 ok>
```

You can write this all on one line. StarForth reads left to right, executing each word as it encounters it:

```
1 ok> 3 4 * .
2 12 ok>
```

Multiplication works the same way. All arithmetic is postfix: the operands come before the operator.

3.4 Inspecting the Stack with .S

.S (dot-S) prints the entire stack without consuming anything. This is your main tool for understanding what is happening:

```
1 ok> 10 20 30
2 ok> .S
3 <3> 10 20 30 ok>
```

The <3> tells you there are three items. The items are shown bottom-to-top, so the rightmost number (30) is on top. The stack is unchanged after .S.

Clear the stack with DROP repeated, or define a word later, or just exit and restart. For now:

```
1 ok> DROP DROP DROP
2 ok> .S
3 <0> ok>
```

3.5 Listing Available Words with WORDS

WORDS prints every word currently in the dictionary:

```
1 ok> WORDS
2 WORD-ENTROPY PHYSICS-FREEZE PHYSICS-THAW WORDS .S .
3 DUP DROP SWAP ROT OVER NIP TUCK 2DUP 2DROP 2SWAP
4 ... (many more) ...
5 ok>
```

The list is long. StarForth ships the full FORTH-79 word set plus extensions. Use WORDS when you cannot remember a word's exact name.

3.6 Defining Your First Word

Use : to define a new word and ; to end the definition:

```
1 ok> : DOUBLE DUP + ;
2 ok> 7 DOUBLE .
3 14 ok>
```

DOUBLE is now in the dictionary. It takes one number, duplicates it, and adds the two copies together. You can use it anywhere.

3.7 Errors and Recovery

Type something StarForth does not recognize:

```
1 ok> FOO
2 Error: word not found: FOO
3 ok>
```

StarForth reports the error and returns to the prompt. The stack is reset to the state it was in before the offending line. You can keep working:

```
1 ok> 3 4 + .
2 7 ok>
```

Stack underflow (trying to pop from an empty stack) is also caught:

```
1 ok> .S
2 <0> ok>
3 ok> DROP
4 Error: stack underflow
5 ok>
```

The system does not crash. Error recovery brings you back to a clean prompt.

3.8 Exiting

Type BYE to exit:

```
1 ok> BYE
```

The VM shuts down cleanly. Any words you defined in the session are gone; nothing is saved automatically. To save work, write it to a source file and load it next time with `INCLUDE` (see Chapter 15).

3.9 Command-Line Shortcuts

You do not have to run interactively. You can pass code directly:

```
1 ./build/amd64/standard/starforth -c "3 4 + . BYE"
2 7
```

Run the test suite and then drop into the REPL:

```
1 ./build/amd64/standard/starforth --run-tests
```


Part II

The Stack

Chapter 4

Stack Fundamentals

FORTH is a stack-based language. Almost everything you do involves the data stack. Understanding it well makes everything else easier.

4.1 What the Stack Is

The data stack is a last-in, first-out (LIFO) sequence of cells. A cell in StarForth is a 64-bit signed integer on a 64-bit host. You push values onto the top of the stack and pop them from the top.

Numbers push automatically. Words consume values (pop) and may produce values (push). The stack effect notation describes what a word does:

(before - after)

Items before the - are consumed (top of stack on the right). Items after are produced. For example, + has the stack effect (a b - a+b): it pops b, pops a, and pushes their sum.

By convention in this guide, the topmost item is labeled n or a; the next item down is n2 or b; and so on.

4.2 Stack Depth

The default data stack depth is 256 cells (configurable at build time via `DATA_STACK_SIZE`). Pushing more than 256 items causes a stack overflow error. Popping from an empty stack causes a stack underflow error. Both are caught and reported; neither crashes the system.

4.3 The Data Stack vs. the Return Stack

StarForth maintains two stacks:

Data stack The main working stack. All arithmetic and most word operations use it. This is the one you interact with directly.

Return stack Manages word call/return linkage and is used as temporary storage with `>R`, `R>`, and `R@`. The return stack is 256 cells deep by default. Do not leave items on the return stack across word boundaries — that causes unpredictable behavior.

Chapter 5

Core Stack Words

This chapter covers the words that manipulate the data stack directly.

5.1 Single-Item Operations

DUP (n - n n)

Duplicates the top of the stack.

```
1 ok> 5 DUP .S
2 <2> 5 5 ok>
```

DROP (n -)

Removes the top of the stack.

```
1 ok> 5 7 DROP .S
2 <1> 5 ok>
```

SWAP (a b - b a)

Exchanges the top two items.

```
1 ok> 1 2 SWAP .S
2 <2> 2 1 ok>
```

ROT (a b c - b c a)

Rotates the top three items: the third item from the top comes to the top.

```
1 ok> 1 2 3 ROT .S
2 <3> 2 3 1 ok>
```

OVER (a b - a b a)

Copies the second item to the top, leaving the original in place.

```
1 ok> 10 20 OVER .S
2 <3> 10 20 10 ok>
```

NIP (a b - b)

Removes the second item (drops below top).

```

1 ok> 10 20 NIP .S
2 <1> 20 ok>

```

TUCK (a b - b a b)

Copies the top item below the second item.

```

1 ok> 10 20 TUCK .S
2 <3> 20 10 20 ok>

```

5.2 Double-Item Operations

These words operate on pairs of items (a “double”), treating two adjacent cells as a single logical unit.

2DUP (a b - a b a b)

Duplicates the top pair.

```

1 ok> 1 2 2DUP .S
2 <4> 1 2 1 2 ok>

```

2DROP (a b -)

Drops the top pair.

```

1 ok> 1 2 3 4 2DROP .S
2 <2> 1 2 ok>

```

2SWAP (a b c d - c d a b)

Exchanges the top two pairs.

```

1 ok> 1 2 3 4 2SWAP .S
2 <4> 3 4 1 2 ok>

```

5.3 The Return Stack

The return stack is normally managed automatically by StarForth’s word-call machinery. You can borrow it as temporary storage within a single word definition using these three words.

>R (n -) (R: - n)

Moves the top of the data stack to the top of the return stack.

R> (- n) (R: n -)

Moves the top of the return stack to the top of the data stack.

R@ (- n) (R: n - n)

Copies (does not move) the top of the return stack to the data stack.

A typical use: stash a loop counter temporarily while performing a computation.

```

1  : EXAMPLE  ( a b -- result )
2      >R      \ stash b on return stack
3      DUP *   \ square a
4      R>      \ retrieve b
5      + ;     \ add b

```

Rule: every >R in a word must be matched by an R> before the word exits, and you must not use >R/R> to move items across loop boundaries (DO LOOP manages the return stack internally).

Part III

Arithmetic and Logic

Chapter 6

Integer Arithmetic

6.1 Basic Arithmetic Words

+ (a b - a+b)

Adds two integers.

```
1 ok> 100 23 + .  
2 123 ok>
```

- (a b - a-b)

Subtracts: removes b from a.

```
1 ok> 100 23 - .  
2 77 ok>
```

*** (a b - a*b)**

Multiplies.

```
1 ok> 7 8 * .  
2 56 ok>
```

/ (a b - a/b)

Integer division; truncates toward zero.

```
1 ok> 17 5 / .  
2 3 ok>
```

MOD (a b - a mod b)

Remainder after division.

```
1 ok> 17 5 MOD .  
2 2 ok>
```

ABS (n - |n|)

Absolute value.

```
1 ok> -42 ABS .
2 42 ok>
```

MIN (a b - min)

Pushes the smaller of two values.

```
1 ok> 10 20 MIN .
2 10 ok>
```

MAX (a b - max)

Pushes the larger of two values.

```
1 ok> 10 20 MAX .
2 20 ok>
```

6.2 Comparison Words

Comparisons push a flag: -1 (all bits set) for true, 0 for false. This is the standard FORTH-79 flag convention.

Word	Stack effect	Meaning
=	(a b - flag)	equal
<>	(a b - flag)	not equal
<	(a b - flag)	a less than b
>	(a b - flag)	a greater than b
<=	(a b - flag)	a less than or equal to b
>=	(a b - flag)	a greater than or equal to b
0=	(n - flag)	true if n is zero
0<	(n - flag)	true if n is negative
0>	(n - flag)	true if n is positive

```
1 ok> 5 3 > .
2 -1 ok>
3 ok> 5 3 < .
4 0 ok>
```

Chapter 7

Logical and Bitwise Operations

7.1 Bitwise Words

In StarForth, the logical words operate bitwise on cells. Because FORTH flags are all-bits-set (-1) for true and zero for false, bitwise AND and OR also work correctly as logical AND and OR when applied to flags.

AND (a b - a&b)

Bitwise AND.

```
1 ok> 0b1100 0b1010 AND .  
2 8 ok>      ( 0b1000 = 8 )
```

OR (a b - a|b)

Bitwise OR.

```
1 ok> 0b1100 0b1010 OR .  
2 14 ok>      ( 0b1110 = 14 )
```

XOR (a b - a xor b)

Bitwise exclusive OR.

```
1 ok> 0b1100 0b1010 XOR .  
2 6 ok>      ( 0b0110 = 6 )
```

INVERT (n - ~n)

Bitwise complement of all bits.

```
1 ok> 0 INVERT .  
2 -1 ok>
```

NOT (n - flag)

Logical NOT: pushes -1 if n is zero, 0 otherwise. Equivalent to 0=.

LSHIFT (n count - n shifted)

Shifts **n** left by **count** bits.

```
1 ok> 1 4 LSHIFT .  
2 16 ok>
```

RSHIFT (n count - n shifted)

Shifts **n** right by **count** bits (logical shift: fills with zeros).

```
1 ok> 256 4 RSHIFT .  
2 16 ok>
```

Chapter 8

Q48.16 Fixed-Point Arithmetic

8.1 What Q48.16 Is

$\mathbb{Q}_{48.16}$ is StarForth’s built-in fixed-point number format. A $\mathbb{Q}_{48.16}$ value is a 64-bit integer where the lower 16 bits represent the fractional part and the upper 48 bits represent the integer part. The precision is $1/65536 \approx 0.0000153$.

You encounter $\mathbb{Q}_{48.16}$ values primarily through the physics runtime (heat values, decay slopes) and in words prefixed with **Q**. In the REPL you will see them printed as rational numbers or as raw integers depending on the display word used.

8.2 When You See Q48.16

The REPL does not automatically distinguish a $\mathbb{Q}_{48.16}$ value from a plain integer — both are cells. Context tells you which you have. The physics diagnostics (Chapter 17) print heat and slope values in $\mathbb{Q}_{48.16}$ form. If you use **WORD-ENTROPY**, expect to see heat counters that may look like large integers; these are $\mathbb{Q}_{48.16}$ values with the fractional part encoded in the low 16 bits.

8.3 Q48.16 Words

The `q48_16_words.c` module provides the Q48.16 word set. Key words:

Word	Stack effect	Description
Q+	(a b - a+b)	Add two $\mathbb{Q}_{48.16}$ values
Q-	(a b - a-b)	Subtract
Q*	(a b - a*b)	Multiply (with precision)
Q/	(a b - a/b)	Divide
INT>Q	(n - q)	Convert integer to $\mathbb{Q}_{48.16}$
Q>INT	(q - n)	Convert $\mathbb{Q}_{48.16}$ to integer (truncate)
Q.	(q -)	Print $\mathbb{Q}_{48.16}$ as decimal fraction

Part IV

Defining Words

Chapter 9

Colon Definitions

9.1 : and ;

The colon word `:` starts a new word definition. Everything that follows — until the matching `;` — becomes the body of the new word:

```
1 : SQUARE ( n -- n2 )  
2   DUP * ;
```

Break this down:

- `:` switches StarForth from interpretation mode to compilation mode.
- *SQUARE* is the name of the new word.
- *DUP* * is the body: duplicate the top, then multiply. The comment in parentheses is documentation only — StarForth ignores it.
- `;` ends the definition and switches back to interpretation mode.

Now use it:

```
1 ok> 9 SQUARE .  
2 81 ok>
```

9.2 A Worked Example

Define a word that computes the sum of squares of two numbers:

```
1 : SQUARE ( n -- n2 ) DUP * ;  
2  
3 : SUM-OF-SQUARES ( a b -- a2+b2 )  
4   SQUARE \ square b (top)  
5   SWAP \ bring a to top  
6   SQUARE \ square a  
7   + ; \ add
```

Test:

```
1 ok> 3 4 SUM-OF-SQUARES .  
2 25 ok>
```

Verify it is in the dictionary:

```
1 ok> WORDS  
2 SUM-OF-SQUARES SQUARE ...  
3 ok>
```

9.3 The SMUDGE Flag

While StarForth is compiling a new word (between `:` and `;`), the word is marked as *smudged*: it exists in the dictionary but is not visible to searches. This prevents a partially-compiled word from being called accidentally during its own definition (unless you explicitly use recursion via `RECURSIVE`). When `;` executes, it clears the smudge flag and the word becomes visible.

If compilation is interrupted by an error, the smudged entry may remain. A subsequent `FORGET` or a new definition with the same name will clean it up.

9.4 IMMEDIATE

Normally, a word defined with `:` is compiled into word bodies when encountered in compilation mode. Marking a word `IMMEDIATE` tells the compiler to execute it at compile time instead:

<pre>1 : MY-WORD ... ; IMMEDIATE</pre>
--

`IF`, `ELSE`, `THEN`, `DO`, `LOOP`, and all the control-flow words are immediate. They do their work (patching branches, emitting addresses) during compilation so the compiled word runs efficiently.

Chapter 10

Variables, Constants, and CREATE

10.1 VARIABLE

VARIABLE creates a named cell in memory:

```
1 VARIABLE COUNTER
```

COUNTER is now a word. Executing it pushes the *address* of the cell (not the value). Use @ to read and ! to write:

```
1 ok> 0 COUNTER !      \ store 0 into COUNTER
2 ok> COUNTER @ .      \ fetch and print
3 0 ok>
4 ok> COUNTER @ 1 + COUNTER !    \ increment
5 ok> COUNTER @ .
6 1 ok>
```

10.2 CONSTANT

CONSTANT creates a word that pushes a fixed value:

```
1 42 CONSTANT ANSWER
```

```
1 ok> ANSWER .
2 42 ok>
```

Unlike a variable, you cannot change the value of a constant after it is defined (without redefining it).

10.3 CREATE and DOES>

CREATE allocates a dictionary entry without specifying behavior. DOES> adds a behavior that runs when the created word is executed. Together they build custom defining words — words that create other words.

A classic example: arrays.

```
1 : ARRAY ( n "name" -- )
2   CREATE CELLS ALLOT
3   DOES> ( index addr -- element-addr )
4   SWAP CELLS + ;
```

```
1 ok> 10 ARRAY MYDATA
2 ok> 99 3 MYDATA !      \ store 99 at index 3
3 ok> 3 MYDATA @ .      \ fetch index 3
4 99 ok>
```

The **DOES>** part becomes the runtime behavior of any word created by **ARRAY**. When **MYDATA** executes, the address of its data area is pushed first, then the **DOES>** code runs with that address on the stack.

Part V

Control Flow

Chapter 11

Conditionals and Loops

Control-flow words are immediate: they compile branches and jumps into the word body. They only work inside word definitions (between `:` and `;`), not in the interactive REPL directly.

11.1 IF ... ELSE ... THEN

IF pops the top of the stack. If it is non-zero (true), the code between IF and ELSE (or THEN) executes. If it is zero, the ELSE branch (if present) executes.

Stack effect of IF: (*flag* -).

```
1 : SIGN ( n -- )
2   DUP 0> IF ." positive" ELSE
3   DUP 0< IF ." negative" ELSE
4   ." zero"
5   THEN
6   THEN
7   DROP ;
```

```
1 ok> 5 SIGN
2 positive ok>
3 ok> -3 SIGN
4 negative ok>
5 ok> 0 SIGN
6 zero ok>
```

Note: IF ... ELSE ... THEN can be nested. Each IF must be matched by exactly one THEN.

11.2 DO ... LOOP

DO starts a counted loop. It pops two items: the limit and the initial index (limit on top). The loop variable I is available inside the loop body.

LOOP increments the index by 1 and exits when the index reaches the limit.

Stack effect of DO: (*limit start* -). Stack effect of I: (- *index*).

```
1 : COUNT-UP ( limit -- )
2   0 DO I . LOOP ;
```

```
1 ok> 5 COUNT-UP
2 0 1 2 3 4 ok>
```

+LOOP increments the index by a specified amount:

```

1 : COUNT-BY-TWO ( limit -- )
2   0 DO I . 2 +LOOP ;

```

```

1 ok> 10 COUNT-BY-TWO
2 0 2 4 6 8 ok>

```

For nested loops, J gives the outer loop index.

LEAVE exits a DO loop immediately:

```

1 : FIND-FIVE ( limit -- )
2   0 DO
3     I 5 = IF ." found 5 at index " I . CR LEAVE THEN
4   LOOP ;

```

11.3 BEGIN ... UNTIL

BEGIN marks the start of a loop. UNTIL pops a flag; if it is non-zero (true), the loop exits; if zero, control returns to BEGIN.

The loop always executes at least once.

```

1 : COUNTDOWN ( n -- )
2   BEGIN
3     DUP .
4     1 -
5     DUP 0=
6   UNTIL
7   DROP ;

```

```

1 ok> 5 COUNTDOWN
2 5 4 3 2 1 ok>

```

11.4 BEGIN ... WHILE ... REPEAT

WHILE is a mid-loop test. WHILE pops a flag; if the flag is zero (false), the loop exits (jumping to after REPEAT); if non-zero, the loop body continues.

```

1 : PRINT-WHILE-POSITIVE ( n -- )
2   BEGIN
3     DUP 0>          \ test: is n positive?
4   WHILE
5     DUP .           \ print n
6     1 -              \ decrement
7   REPEAT
8   DROP ;

```

```

1 ok> 4 PRINT-WHILE-POSITIVE
2 4 3 2 1 ok>

```

11.5 A Note on Interactive Compilation

Control-flow words like IF and DO compile branch addresses that only make sense inside a complete word definition. Typing them directly at the ok> prompt will produce an error. Always wrap control flow inside : ... ;.

Part VI

Memory and Blocks

Chapter 12

Memory Access

12.1 Cell-Sized Access: @ and !

@ (fetch) reads one cell (64 bits on a 64-bit host) from an address. ! (store) writes one cell to an address.

@ (addr - n)

Fetch the cell at addr:

```
1 ok> VARIABLE X
2 ok> 42 X !
3 ok> X @ .
4 42 ok>
```

! (n addr -)

Store n at addr:

```
1 ok> 99 X !
2 ok> X @ .
3 99 ok>
```

12.2 Byte-Sized Access: C@ and C!

C@ and C! read and write a single byte (character).

C@ (addr - char)

Fetch one byte.

C! (char addr -)

Store one byte.

These are used for string manipulation and working with character buffers.

12.3 Bulk Memory: MOVE and FILL

MOVE (src dst count -)

Copies count bytes from src to dst. Handles overlapping regions correctly.

FILL (addr count char -)

Fills `count` bytes starting at `addr` with the byte value `char`.

Chapter 13

The Block Subsystem

13.1 What Blocks Are

FORTH-79 uses a block-oriented storage model. A block is 1024 bytes of data, addressed by a logical block number (LBN). Blocks provide a uniform interface to storage whether the backing medium is a file, RAM, or physical disk.

In StarForth the block subsystem maps logical block numbers to physical storage through a pluggable backend (file, RAM, or factory). The default configuration uses RAM blocks for interactive sessions.

13.2 The VM Memory Map

StarForth allocates 5 MB of virtual memory for the VM:

Region	LBN range	Size
Dictionary (word definitions)	LBN 0–2047	2 MB
User blocks	LBN 2048 and above	user data
Log blocks	LBN 3072–5120	2 MB

`init.4th` is loaded from LBN 2048 at boot in the bare-metal kernel configuration. On hosted Linux, `init.4th` is loaded from the filesystem.

13.3 Block Words

BLOCK (lbn - addr)

Finds or loads block `lbn` into a buffer and pushes the buffer address. The block is now in memory and can be read with `@ / C@` or written with `! / C!`.

BUFFER (lbn - addr)

Like **BLOCK** but does not read from storage: it claims a buffer for `lbn` and leaves its contents uninitialized. Use when you intend to write the entire block and do not need to read it first.

LOAD (lbn -)

Loads and executes the FORTH source in block `lbn`. This is the traditional FORTH way to run stored code.

THRU (lbn1 lbn2 -)

Loads and executes blocks **lbn1** through **lbn2** inclusive.

FLUSH (-)

Writes all dirty buffers back to storage. Use before exiting or before switching tasks if you have written to blocks.

UPDATE (-)

Marks the most recently referenced block buffer as dirty, so **FLUSH** will write it back.

Chapter 14

String and I/O Words

14.1 Output Words

EMIT (**char** -)

Outputs one character (given as an ASCII code).

```
1 ok> 65 EMIT
2 A ok>
```

CR (-)

Outputs a newline (carriage return and line feed).

SPACE (-)

Outputs a single space.

TYPE (**addr count** -)

Outputs **count** characters starting at **addr**.

.((**"text"** -)

Immediately prints the text that follows (up to the closing parenthesis). Works in both interpretation and compilation mode.

```
1 ok> .( Hello, world!)
2 Hello, world! ok>
```

." (**"text"** -)

Inside a word definition, compiles a string literal that prints when the word executes:

```
1 : GREET ." Hello from StarForth!" CR ;
```

14.2 Input Words

KEY (- **char**)

Waits for a keypress and pushes the ASCII code.

ACCEPT (addr maxcount - count)

Reads a line of input into the buffer at **addr**, up to **maxcount** characters. Returns the actual count of characters read.

14.3 String Literals: S"

S" compiles a string. At runtime it pushes the address and count of the string data:

```
1  : PRINT-NAME  S" StarForth" TYPE CR ;
```

```
1  ok> PRINT-NAME
2  StarForth
3  ok>
```

Chapter 15

Running Files and Blocks

15.1 INCLUDE

INCLUDE loads a FORTH source file from the filesystem and executes it:

```
1 ok> S" myprogram.4th" INCLUDE
2 ok>
```

All words defined in the file are added to the dictionary. Errors in the included file are reported with the filename and line number.

15.2 EXEC

EXEC loads a named capsule file (the StarForth extension for loading .4th capsule files by name):

```
1 S" _capsule.4th" EXEC
```

15.3 LOAD and THRU

LOAD and THRU execute source from block storage (see Chapter [13](#)).

Chapter 16

The Interactive REPL

16.1 The `ok>` Prompt

The `ok>` prompt signals that StarForth is in interpretation mode, the data stack is not empty but no error has occurred, and the system is waiting for input. More precisely:

- `ok>` — normal prompt; data stack may or may not be empty.
- After an error — StarForth prints the error, clears the stack to the pre-error state, and re-prints `ok>`.

16.2 Printing Numbers

The basic output words:

Word	Stack effect	Effect
<code>.</code>	(<code>n</code> -)	Print top of stack in current base, then space
<code>.S</code>	(-)	Print entire stack without consuming it
<code>.R</code>	(<code>n width</code> -)	Print right-justified in field of width

16.3 Numeric Base

StarForth defaults to decimal (base 10). Switch with:

```
1 ok> HEX
2 ok> FF .
3 255 ok>
4 ok> DECIMAL
5 ok>
```

`BASE` is a variable that holds the current numeric base (2–36). `HEX` stores 16 in `BASE`; `DECIMAL` stores 10.

16.4 Dictionary Introspection

Word	Stack effect	Description
<code>WORDS</code>	(-)	Print all dictionary words
<code>FIND</code>	(<code>addr len</code> - <code>xt flags</code> 0)	Search by name
<code>'</code>	(<code>"name"</code> - <code>xt</code>)	Push execution token of named word
<code>SEE</code>	(<code>"name"</code> -)	Decompile a word

Part VII

Physics Features

Chapter 17

The Adaptive Runtime

17.1 What the Adaptive Runtime Does for You

While you type and run FORTH code, StarForth’s adaptive runtime is watching in the background. It tracks which words you execute most often, maintains a rolling history of execution sequences, and adjusts internal caches and decay rates to keep hot words fast and cold words from consuming resources.

From a user’s perspective, you do not need to configure any of this. The system tunes itself. The physics features become relevant when you want to observe what the runtime is doing (`WORD-ENTROPY`), when you want to lock behavior in place for reproducible benchmarks (`PHYSICS-FREEZE/PHYSICS-THAW`), or when you run the Design of Experiments mode (`-doe`).

The runtime uses thermodynamic language as a modeling convenience: “heat” is execution frequency, “decay” is the process of cooling quiescent words, “stability” is a statistical property of the heat trajectory. These are descriptive terms for internal metrics, not claims about physical thermodynamics.

17.2 The Seven Feedback Loops

The adaptive runtime is governed by seven feedback loops running concurrently. Here is a brief description of what each one does. For full technical detail, see Volume I of the StarForth VM and Physics Runtime reference.

17.3 The Seven Feedback Loops

StarForth’s adaptive runtime is governed by seven configurable feedback loops. Each loop pairs a positive (accumulation) signal that drives the runtime toward optimization against a negative (stabilization) signal that prevents runaway growth and holds the system at equilibrium. Every loop can be toggled independently through a Makefile flag of the form `ENABLE_LOOP_N_*`, and the loops are designed to compose cumulatively so that Design of Experiments (DoE) campaigns can measure each increment in isolation.

The loops borrow thermodynamic language as a modeling convenience. Using execution frequency as a proxy for thermal energy, the runtime tracks “heat” per word, lets it “decay” over time, and reasons about “stability” in statistical terms. These are descriptive tools, not claims of physical thermodynamics.

17.3.1 Loop #1 — Execution Heat Tracking

Loop #1 counts word usage to identify hot execution paths. Every execution increments `DictEntry.execution_heat`, and words crossing the promotion threshold migrate into the

Loop	Name	Accumulation signal	Stabilization signal
#1	Execution Heat	word execution increments heat	Loop #3 decay; cold words demoted
#2	Rolling Window	every execution recorded to buffer	wrap overwrite; adaptive shrinking
#3	Linear Decay	slope starts at 1/3	decay reduces heat; slope adjusts $\pm 5\%$
#4	Pipelining	word→word transitions recorded	counts age out; miss-rate feedback
#5	Window Inference	window grows with diversity	Levene’s test shrinks to minimum
#6	Decay Inference	regression extracts decay rate	ANOVA early-exit; fit-quality bound
#7	Adaptive Heartrate	tick counter increments	inference cadence falls when stable

Table 17.1: The seven feedback loops, with their accumulation and stabilization mechanisms.

`hotwords_cache` for constant-time lookup. Stabilization comes from Loop #3, which decays heat over time, and from cache demotion: words falling below `HEAT_CACHE_DEMOTION_THRESHOLD` (10) are evicted. The `PHYSICS-RESET-STATS` word clears all heat counters. Build flag: `ENABLE_LOOP_1_HEAT_TRACKING`.

17.3.2 Loop #2 — Rolling Window History

Loop #2 captures the execution sequence in a circular buffer (`execution_history`) to seed metrics deterministically. The window grows to `ROLLING_WINDOW_SIZE` (default 4096) before wrapping and becomes “warm” after 1024 executions, the point at which it holds representative data. Stabilization is twofold: oldest entries are overwritten on wrap, and adaptive shrinking reduces `effective_window_size` whenever pattern-diversity growth falls below `ADAPTIVE_GROWTH_THRESHOLD` (1%). The check runs every `ADAPTIVE_CHECK_FREQUENCY` (256) executions, retains `ADAPTIVE_SHRINK_RATE` (75%) of the window per cycle, and never shrinks below `ADAPTIVE_MIN_WINDOW_SIZE` (256). Build flag: `ENABLE_LOOP_2_ROLLING_WINDOW`.

17.3.3 Loop #3 — Linear Decay

Loop #3 ages words to prevent unbounded heat accumulation; it is primarily a negative-feedback loop. Heat decays at `DECAY_RATE_PER_US_Q16` (1/65536 heat per microsecond), giving a half-life of roughly six to seven seconds for a 100-heat word. The decay slope, stored as a $\mathbb{Q}_{48.16}$ value (`decay_slope_q48`, initialized to 1/3), self-adjusts by $\pm 5\%$ against the hot-to-stale word ratio: an excess of hot words increases the slope, an excess of stale words decreases it. A minimum interval `DECAY_MIN_INTERVAL` (1 μ s) guards against over-decay. Build flag: `ENABLE_LOOP_3_LINEAR_DECAY`.

17.3.4 Loop #4 — Pipelining Metrics

Loop #4 tracks word-to-word transitions to enable speculative prefetch. `WordTransitionMetrics` records transitions per word and builds context windows of depth `TRANSITION_WINDOW_SIZE` (default 2) for prediction, while `prefetch_attempts` and `prefetch_hits` track accuracy. Transition counts age out over time, and a miss-rate signal drives a binary-chop search over window size through `PipelineGlobalMetrics.suggested_next_size`. Build flag: `ENABLE_LOOP_4_PIPELINING_METRICS`.

17.3.5 Loop #5 — Window Width Inference

Loop #5 finds the optimal rolling-window size through statistical testing. The `effective_window_size` expands toward `ROLLING_WINDOW_SIZE` when pattern diversity rises above threshold. For stabilization it applies Levene’s test for equality of variance: the heat trajectory is split into K disjoint chunks, each chunk’s variance is computed independently, and the test statistic W is compared against the critical value ($W \approx 6.5$ at $\alpha = 0.05$). When $W \leq$ critical, the variances are stable and the window has reached its minimum sufficient size, preventing over-capture of redundant pattern data. Build flag: `ENABLE_LOOP_5_WINDOW_INFERENCE`.

17.3.6 Loop #6 — Decay Slope Inference

Loop #6 extracts the optimal decay rate by exponential regression over the heat trajectory. Fitting the log-linear model

$$\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} \cdot t \quad (17.1)$$

yields a closed-form `adaptive_decay_slope`. An ANOVA early-exit guards the cost: if `has_variance_stabilized()` reports that variance changed by less than 5% since the last inference, the computation is skipped and the cached result reused, saving an estimated 5–10k CPU cycles. The slope is bounded by fit quality (`slope_fit_quality_q48`), and `inference_outputs_validate()` rejects invalid slopes (zero or greater than 100). Build flag: `ENABLE_LOOP_6_DECAY_INFERENCE`.

17.3.7 Loop #7 — Adaptive Heartrate

Loop #7 adjusts tick frequency to balance responsiveness against inference cost. `tick_count` increments every heartbeat; the heartbeat thread wakes at `HEARTBEAT_TICK_NS` (1 ms default) and samples hot-word count, total heat, and window width. When the system is stable, full inference runs less often, gated by `HEARTBEAT_INFERENCE_FREQUENCY` (5000 ticks): expensive inference triggers a slower tick cadence. The background `HeartbeatWorker` thread can be disabled entirely with `HEARTBEAT_THREAD_ENABLED=0`. Build flag: `ENABLE_LOOP_7_ADAPTIVE_HEARTRATE`.

17.3.8 Principal Stabilizers

Five mechanisms carry the bulk of the system’s negative feedback:

- **Heat decay** (Loop #3) — the primary stabilizer, preventing unbounded heat accumulation.
- **Window shrinking** (Loops #2 and #5) — a diversity plateau triggers size reduction via Levene’s test.
- **ANOVA early-exit** (Loop #6) — variance stability below 5% change skips expensive inference.
- **Cache demotion** (Loop #1) — words below the demotion threshold leave the hot-words cache.
- **Slope reversal** (Loop #3) — an inverted hot-to-stale ratio adjusts the decay slope by $\pm 5\%$.

17.3.9 Configuration

Any loop can be disabled at build time. Setting all seven flags to zero yields the baseline configuration used as the DoE reference point.

```

1  # Disable a specific loop
2  make ENABLE_LOOP_3_LINEAR_DECAY=0
3
4  # Baseline build (all loops off)
5  make ENABLE_LOOP_1_HEAT_TRACKING=0 \
6        ENABLE_LOOP_2_ROLLING_WINDOW=0 \
7        ENABLE_LOOP_3_LINEAR_DECAY=0 \
8        ENABLE_LOOP_4_PIPELINING_METRICS=0 \
9        ENABLE_LOOP_5_WINDOW_INFERENCE=0 \
10       ENABLE_LOOP_6_DECAY_INFERENCE=0 \
11       ENABLE_LOOP_7_ADAPTIVE_HEARTRATE=0

```

Knob	Default	Description
ROLLING_WINDOW_SIZE	4096	Initial/maximum window size
ADAPTIVE_SHRINK_RATE	75	% retained when shrinking
ADAPTIVE_MIN_WINDOW_SIZE	256	Floor for window shrinking
ADAPTIVE_CHECK_FREQUENCY	256	Executions between diversity checks
ADAPTIVE_GROWTH_THRESHOLD	1	Growth rate (%) signalling saturation
DECAY_RATE_PER_US_Q16	1	Heat decay per microsecond ($\mathbb{Q}_{48.16}$)
HEARTBEAT_INFERENCE_FREQUENCY	5000	Ticks between full inference runs
TRANSITION_WINDOW_SIZE	2	Context depth for pipelining prediction

Table 17.2: Tuning knobs for the feedback loops.

The implementation is distributed across `src/rolling_window_of_truth.c` (Loop #2), `src/dictionary_heat_optimization.c` (Loops #1 and #3), `src/inference_engine.c` (unified Loops #5 and #6), `src/physics_pipelining_metrics.c` (Loop #4), and the `vm_tick()` heartbeat dispatcher in `src/vm.c` (Loop #7). Knob definitions live in `include/rolling_window_knobs.h`.

17.4 WORD-ENTROPY

WORD-ENTROPY prints execution heat statistics for every word in the dictionary to stdout. Use it to see which words are running hot:

```
1 ok> WORD-ENTROPY
```

The output lists each word with its current execution heat (a $\mathbb{Q}_{48.16}$ value), the number of times it has been called, and its position in the hot-words cache (if it is cached). Words with zero heat have not been called since the last reset.

WORD-ENTROPY is intentionally kept active in DoE mode because the diagnostic value outweighs the output noise.

17.5 PHYSICS-FREEZE and PHYSICS-THAW

PHYSICS-FREEZE suspends all adaptive runtime updates. Heat values stop changing, the decay loop stops, and the hot-words cache is locked in its current state. Everything else — word execution, I/O, compilation — continues normally.

PHYSICS-THAW resumes adaptive updates from the point they were frozen.

Use these words when you want reproducible timing:

```
1 ok> PHYSICS-FREEZE
2 ok> \ ... run your benchmark here ...
3 ok> PHYSICS-THAW
```

You might also freeze before a sensitive section to prevent cache eviction from perturbing timing, then thaw afterward.

17.6 PHYSICS-RESET-STATS

PHYSICS-RESET-STATS clears all heat counters back to zero. Useful before a fresh measurement run:

```
1 ok> PHYSICS-RESET-STATS
2 ok> \ ... warm up and then measure ...
```

Chapter 18

Heartbeat and Adaptive Tuning

18.1 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

18.1.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
1 tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
   bucket_hits_delta,
2 word_executions_delta,hot_word_count,avg_word_heat>window_width,
3 predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

18.1.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
1 # Capture metrics, discard test output
2 ./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null
3
4 # Keep both streams, separated
5 ./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
   csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
1 { echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
2 bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
   ,\
```

Table 18.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
<code>tick_number</code>	<code>uint32</code>	count	Monotonic tick counter since VM start	live
<code>elapsed_ns</code>	<code>uint64</code>	ns	Time since VM initialization	live
<code>tick_interval_ns</code>	<code>uint64</code>	ns	Time since previous tick (actual vs. nominal 1 ms)	live
<code>cache_hits_delta</code>	<code>uint32</code>	count	Hotwords cache hits since last tick	TODO
<code>bucket_hits_delta</code>	<code>uint32</code>	count	Bucket-level cache hits since last tick	TODO
<code>word_executions_delta</code>	<code>uint32</code>	count	FORTH word executions since last tick	TODO
<code>hot_word_count</code>	<code>uint64</code>	count	Words at or above the heat promotion threshold (≥ 10)	live
<code>avg_word_heat</code>	<code>double</code>	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \rightarrow \text{double}$)	live
<code>window_width</code>	<code>uint32</code>	entries	Current rolling window effective size (adaptive)	live
<code>predicted_label_hits</code>	<code>uint32</code>	count	Successful pipelining speculation hits	TODO
<code>estimated_jitter_ns</code>	<code>double</code>	ns	Absolute deviation from the nominal 1 ms interval	live

```

3 window_width,predicted_label_hits,estimated_jitter_ns"; \
4 cat heartbeat.csv; } > analysis.csv

```

18.1.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

18.1.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

18.1.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 19

Design of Experiments Mode

19.1 The `-doe` Flag

The `-doe` flag runs the full test harness in DoE (Design of Experiments) mode and then exits:

```
1 ./build/amd64/fastest/starforth --doe
```

DoE mode exercises all 936+ tests and all seven feedback loops. It is used for systematic experimental runs comparing different build configurations.

19.2 Capturing Heartbeat Metrics

In DoE mode, heartbeat metrics stream to `stderr`. Capture them for analysis:

```
1 ./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null
```

See Chapter 18 for the CSV format and analysis guidance.

19.3 The 2^7 Factorial Design

DoE uses a 2^7 factorial design, one factor per feedback loop. Each run enables a specific combination of loops; the metrics measure the incremental effect of each loop addition. The reference (baseline) has all seven loops disabled. At the opposite extreme, all seven loops active, the system is fully adaptive.

19.4 The Built-In Profiler

19.5 The Built-In Word Profiler

StarForth ships a lightweight word-execution profiler that tracks how often each word runs and turns that data into optimization guidance. Its purpose is to surface hot paths — frequently executed words — that are candidates for inline assembly. At its basic level the profiler adds well under 1% overhead, making it suitable for always-on use.

19.5.1 Quick Start

Profiling is enabled with two flags: `-profile` sets the detail level and `-profile-report` prints the report on exit.

```
1 ./build/starforth --profile 1 --profile-report < script.fth
```

The report lists global statistics and the most frequently called words by execution count — typically dominated by `LIT`, `EXIT`, `(LOOP)`, `I`, and the core stack and arithmetic primitives.

19.5.2 Profiling Levels

Level	Overhead	Adds
0 — <code>PROFILE_DISABLED</code>	0%	Nothing (production default)
1 — <code>PROFILE_BASIC</code>	<1%	Frequency & lookup counts
2 — <code>PROFILE_DETAILED</code>	5–10%	Per-word timing (ns)
3 — <code>PROFILE_VERBOSE</code>	15–20%	Stack & memory access tracking

Table 19.1: Profiling levels and their cost.

Basic profiling tracks word frequency and dictionary lookups at negligible cost. Detailed profiling adds nanosecond-precision execution timing with average, minimum, and maximum per word. Verbose profiling further records stack operation counts and memory read/write volumes.

19.5.3 Reading the Report

High call counts mark hot paths. The detailed view reports total time per word in microseconds alongside average and maximum nanoseconds, exposing words that are individually slow as well as those that are merely frequent. A typical workload concentrates 80–95% of all executions in its top ten words, so a small set of targeted optimizations affects nearly all runtime.

19.5.4 Hot-Word Analysis

The `profiler_print_hotspots()` routine ranks words by share of total executions and attaches a recommendation to each.

Share of total	Recommendation
$\geq 5.0\%$	High priority — inline-assembly candidate
$\geq 2.0\%$	Consider assembly optimization
$\geq 1.0\%$	Monitor for optimization
$\geq 0.5\%$	Defer to profile-guided optimization
$< 0.5\%$	Low priority

Table 19.2: Optimization priority by execution share.

19.5.5 Development Workflow

The intended cycle is: profile a representative workload at the basic level, identify hot words, then optimize. Words above 5% of executions warrant a hand-written assembly path under a `USE_ASM_OPT` guard; words in the 0.5–5% band are better left to profile-guided optimization, where the compiler inlines and arranges them from the same data. Gains are confirmed by benchmarking before and after with `-benchmark`.

Best practice is to profile real programs rather than toy examples, run enough iterations for statistical significance (1000+ word executions), and start at the basic level. Profiling should avoid `LOG_DEBUG` (logging overhead skews results), focus on normal rather than error paths, and treat a top-ten coverage below 80% as a sign the workload is unrepresentative.

19.5.6 Implementation

The profiler instruments execution in two places: the outer interpreter (`vm_interpret_word()`), which sees words run directly from the REPL or scripts, and the inner interpreter (`execute_colon_word()`), which sees words run from compiled threaded code. Per-word statistics are held in a compact record:

```

1  typedef struct {
2      const DictEntry *entry;      // dictionary entry
3      uint64_t call_count;         // executions
4      uint64_t total_time_ns;      // DETAILED+
5      uint64_t min_time_ns;        // DETAILED+
6      uint64_t max_time_ns;        // DETAILED+
7  } WordStats;

```

Frequency tracking is a single counter increment with $O(1)$ lookup for known entries and $O(n)$ only on a word's first call, with capacity for 256 unique words (expandable; primitive words are always tracked). It performs no timing calls, which is what keeps the basic level effectively free.

19.5.7 C API

```

1  int  profiler_init(ProfileLevel level);
2  void profiler_shutdown(void);
3  void profiler_word_count(const DictEntry *entry);    // lightweight
4  void profiler_word_enter(const DictEntry *entry);    // timing
5  void profiler_word_exit(const DictEntry *entry);
6  void profiler_generate_report(void);
7  void profiler_print_hotspots(void);
8  void profiler_reset(void);

```

19.5.8 Integration with External Tools

The built-in profiler complements rather than replaces system profilers. It reports hot *Forth* words; `perf` reports hot *C* functions; Valgrind/Callgrind supplies exact instruction counts; and `gprof` provides function-level timing. The recommended combination runs the StarForth profiler at the basic level continuously and reaches for the detailed level or an external tool only for targeted investigation. Typical failures are simple: no data usually means one of the two flags is missing, all-zero counts mean no Forth code executed before `BYE`, excessive overhead means a level above basic is in use, and missing words mean the 256-entry capacity was reached.

Part VIII

The ACL System

Chapter 20

Access Control for Interactive Users

20.1 What ACL Means for You

StarForth has a word-level access-control system. In a default development build the ACL system is disabled (security is opt-in), so you will not encounter it in most sessions.

When the ACL system is active, individual dictionary words may be restricted. Attempting to execute a restricted word produces an error. Which words are restricted, and under what conditions, is determined by the ACL policy loaded at boot.

Most users on a standard build will never see ACL restrictions. The feature is intended for deployments where StarForth serves as a controlled execution environment (embedded systems, multi-user shells, kernel contexts).

20.2 The `ok>` Prompt vs. Emergency `ok>`

There are two permanent console layers in StarForth:

Normal REPL — `ok>` The standard interactive prompt. ACL restrictions are in effect if the ACL system is active.

Emergency console — `ok>` When a fault handler activates (a crash, an unrecoverable error), StarForth falls back to the emergency console. The emergency console has a 100% ACL bypass — all words execute regardless of their ACL state. This ensures you can always recover, even in a locked-down deployment.

The `EMERGENCY_CONSOLE_ENABLED` build flag (default: on) controls whether the emergency console is present. In production embedded builds it may be disabled.

20.3 The Zuse Session

`zuse` is the bootstrap superuser in StarForth's security model. When a Zuse session is active, the prompt changes to:

```
1  zuse)ok>
```

A Zuse session has a full ACL bypass: all words execute regardless of their normal ACL state. The Zuse console is completely independent of the emergency console — either may be active without affecting the other.

How to authenticate as Zuse and what operations a Zuse session enables is covered in the StarForth Developer Guide, not here. From a user's perspective, if you see `zuse)ok>`, you are in a privileged session.

20.4 Enabling the ACL System

The ACL system is opt-in. In `init.4th`, one commented-out line controls it:

```
1 \ S" ACL.4th" EXEC
```

Uncommenting that line enables full ACL enforcement at boot. Leaving it commented keeps a development build open. See the StarForth Developer Guide for the full ACL design.

Part IX

FORTH Programming

Chapter 21

Thinking in FORTH

FORTH is a different kind of language. If you come from C, Python, or Java, a few shifts in thinking will make it click faster.

21.1 Postfix Notation

Every language you have probably used puts operators between operands: `3 + 4`. FORTH puts the operands first and the operator last: `3 4 +`. This is postfix (or reverse Polish) notation.

The reason is the stack. When you type `3 4 +`, the runtime pushes 3, pushes 4, then `+` pops both and pushes 7. There are no parentheses for grouping because order of operations is completely explicit. Compare:

Infix (C)	FORTH
<code>(3 + 4) * 2</code>	<code>3 4 + 2 *</code>
<code>a * a + b * b</code>	<code>a a * b b * +</code>
<code>(x - 1) / (x + 1)</code>	<code>x 1 - x 1 + /</code>

You evaluate left to right, and the result of each operation is immediately available for the next.

21.2 Words Are Verbs

In FORTH, a “word” is any named operation — primitive or user-defined. Numbers push themselves; everything else is a word that does something to the stack. `DUP` is a word. `+` is a word. `HELLO` (once you define it) is a word.

This uniformity means the language is extensible in a deep way. After you define `SQUARE`, it is indistinguishable from a primitive — it appears in `WORDS`, it runs when you type it, and other words can call it. The line between “built-in” and “user-defined” is stylistic, not structural.

21.3 Factor Early, Factor Often

The FORTH style is to decompose programs into many small words, each doing one thing. A word with more than five or six lines in its body is usually a sign that it wants to be split.

```
1  ( Bad: one big word )
2  : PROCESS-PAYMENT ( amount account-id -- )
3      DUP VALID-ACCOUNT? IF
4          OVER CHECK-FUNDS IF
5              SWAP DEDUCT
```

```

6      LOG-TRANSACTION
7      ELSE ." Insufficient funds" CR THEN
8      ELSE ." Invalid account" CR THEN
9      DROP ;

1 ( Better: factored )
2 : REJECT-INVALID ." Invalid account" CR DROP DROP ;
3 : REJECT-FUNDS   ." Insufficient funds" CR DROP DROP ;
4 : COMMIT-PAYMENT ( amount account-id -- )
5     SWAP DEDUCT LOG-TRANSACTION ;
6
7 : PROCESS-PAYMENT ( amount account-id -- )
8     DUP VALID-ACCOUNT? NOT IF REJECT-INVALID EXIT THEN
9     OVER CHECK-FUNDS NOT IF REJECT-FUNDS EXIT THEN
10    COMMIT-PAYMENT ;

```

Factored code is easier to test interactively, easier to reuse, and easier to read. You can test COMMIT-PAYMENT in isolation before wiring it into PROCESS-PAYMENT.

21.4 Stack Comments Are Documentation

FORTH comments use parentheses: (*this is a comment*). By convention, every word definition begins with a stack-effect comment:

```

1 : AREA ( width height -- area )
2     * ;

```

The stack comment is the primary documentation. Write it first, before the body, and treat it as a contract: the word must consume exactly the items before the `-` and produce exactly the items after. Violations cause hard-to-debug stack confusion.

StarForth does not enforce stack effects at runtime — it is your responsibility to make them accurate.

21.5 The Interactive Development Loop

FORTH's interactive nature is a feature, not a limitation. The typical development cycle is:

1. Type a small fragment at the `ok>` prompt and test it.
2. When it works, wrap it in a `: NAME ... ;` definition.
3. Test the word interactively.
4. Build larger words on top of working smaller ones.

For example, building a Fahrenheit-to-Celsius converter step by step:

```

1 ok> 32 .
2 32 ok>
3
4 ok> 212 32 - .
5 180 ok>
6
7 ok> 212 32 - 5 * 9 / .
8 100 ok>
9
10 ok> : F>C ( f -- c ) 32 - 5 * 9 / ;

```

```
11 | ok> 212 F>C .  
12 | 100 ok>  
13 | ok> 32 F>C .  
14 | 0 ok>  
15 | ok> 98 F>C .  
16 | 36 ok>
```

Each step confirmed before moving on.

Chapter 22

Common Programming Patterns

22.1 Accumulator Pattern

Use a variable as an accumulator and update it in a loop:

```
1 VARIABLE SUM
2
3 : SUM-TO ( n -- sum )
4   0 SUM !
5   1 + 1 DO
6     I SUM @ + SUM !
7   LOOP
8   SUM @ ;
```

```
1 ok> 10 SUM-TO .
2 55 ok>
```

The pattern: initialize the variable before the loop, update inside, read after.

22.2 Table Dispatch

Use EXECUTE to implement a jump table. First, build an array of execution tokens:

```
1 : GREET-EN ." Hello" ;
2 : GREET-ES ." Hola" ;
3 : GREET-FR ." Bonjour" ;
4
5 CREATE GREET-TABLE
6   ' GREET-EN ,
7   ' GREET-ES ,
8   ' GREET-FR ,
9
10 : GREET ( lang-index -- )
11   CELLS GREET-TABLE + @ EXECUTE CR ;
```

```
1 ok> 0 GREET
2 Hello ok>
3 ok> 1 GREET
4 Hola ok>
5 ok> 2 GREET
6 Bonjour ok>
```

CELLS converts the index to a byte offset. EXECUTE runs the stored execution token.

22.3 Counted String Pattern

A common FORTH idiom is the counted string: a byte at the start holds the length, followed by the characters. `COUNT` converts a counted-string address to the `( addr len )` form that `TYPE` expects:

```
1 : PRINT-COUNTED ( c-addr -- )
2   COUNT TYPE CR ;
```

`S"` produces an `( addr len )` pair directly:

```
1 ok> S" hello world" TYPE CR
2 hello world
3 ok>
```

22.4 Guard Clauses

Use `EXIT` at the start of a word to reject bad inputs early. This is cleaner than deeply nested `IF`s:

```
1 : SAFE-DIVIDE ( a b -- a/b )
2   DUP 0= IF
3     ." Division by zero" CR
4     DROP DROP 0
5     EXIT
6   THEN
7   / ;
```

```
1 ok> 10 2 SAFE-DIVIDE .
2 5 ok>
3 ok> 10 0 SAFE-DIVIDE .
4 Division by zero
5 0 ok>
```

22.5 State Machine in FORTH

Encode state as an integer in a variable, and dispatch on it:

```
1 0 CONSTANT STATE-IDLE
2 1 CONSTANT STATE-RUN
3 2 CONSTANT STATE-DONE
4
5 VARIABLE MACHINE-STATE
6 STATE-IDLE MACHINE-STATE !
7
8 : TRANSITION ( new-state -- ) MACHINE-STATE ! ;
9 : STATE@      ( -- state )      MACHINE-STATE @ ;
10
11 : STEP
12   STATE@ CASE
13     STATE-IDLE OF ." Idle -> Run"  STATE-RUN TRANSITION ENDOF
14     STATE-RUN  OF ." Running..." STATE-DONE TRANSITION ENDOF
15     STATE-DONE OF ." Done."
16   ENDOF
17   ENDCASE
18   CR ;
```

`CASE` / `OF` / `ENDOF` / `ENDCASE` are the FORTH multi-way branch words.

Chapter 23

Debugging at the Prompt

23.1 Using .S Liberally

.S is non-destructive — it leaves the stack intact. Sprinkle it freely while working out what a piece of code does:

```
1 ok> 5 7 .S * .S .
2 <2> 5 7
3 <1> 35
4 35 ok>
```

You can also insert .S inside a word definition during debugging (it compiles to a call to the runtime print-stack routine):

```
1 : BUGGY ( a b -- result )
2   .S           \ what do we have?
3   +
4   .S           \ what did + produce?
5   2 * ;
```

Remove the .S calls once the word works correctly.

23.2 SEE: Decompiling a Word

SEE decompiles a word, showing its compiled form:

```
1 ok> SEE DOUBLE
2 : DOUBLE DUP + ; ok>
```

The output is a reconstruction from the compiled threaded code. It may not look identical to the original source but shows the same operations.

23.3 WORDS: Finding What Exists

When you cannot remember whether you defined a word:

```
1 ok> WORDS
```

The output is long. You can pipe StarForth output through **grep** at the shell level:

```
1 echo "WORDS_BYE" | ./build/amd64/standard/starforth | grep SQUARE
```

23.4 Common Mistakes

Wrong argument order. Postfix is different from most calculators. `10 3 -` gives 7, not `-7`. When in doubt, use `.S` before and after each operation.

Forgetting to consume values. If `.S` shows values you did not expect, a word left extra values on the stack. Check every word's documented stack effect.

Control flow outside a definition. `IF`, `DO`, `BEGIN`, etc. are compile-only. Typing them at the prompt produces an error. Wrap them inside `: ... ;`.

Mismatched `IF` / `THEN`. Every `IF` needs a `THEN`. Forgetting one leaves the compiler in a confused state. An error mid-definition usually means the definition was not completed; start over with a fresh `:` definition.

Return stack imbalance. Every `>R` in a word must be matched by an `R>` before `;`. Leaving items on the return stack corrupts the call chain.

Chapter 24

Working with Numbers

24.1 Integer Ranges

A StarForth cell is a 64-bit signed integer on a 64-bit host. The range is -2^{63} to $2^{63} - 1$ (approximately $\pm 9.2 \times 10^{18}$). Overflow wraps silently (two's complement arithmetic) — StarForth does not trap integer overflow by default.

```
1 ok> 9223372036854775807 1 + .  
2 -9223372036854775808 ok>
```

That is the maximum signed 64-bit integer wrapping to the minimum. Keep this in mind when working with large values.

24.2 Numeric Literals

StarForth accepts numbers in the current base. In decimal (the default):

```
1 ok> 255 .  
2 255 ok>
```

After HEX:

```
1 ok> HEX  
2 ok> FF .  
3 255 ok>  
4 ok> DECIMAL
```

Negative numbers use the minus sign prefix:

```
1 ok> -7 ABS .  
2 7 ok>
```

24.3 Double-Cell Numbers

When you need values larger than one cell, FORTH uses double-cell numbers — two consecutive cells treated as a single 128-bit value (on a 64-bit host). Words that produce or consume double-cells are conventionally prefixed with D (e.g. D+, D-) or suffixed with D in their stack comments. A double cell occupies two stack positions: the lower (less significant) cell on top.

M* multiplies two single cells and produces a double cell result:

```
1 ok> 1000000 1000000 M* D. CR  
2 1000000000000  
3 ok>
```

Double-cell arithmetic is used when single-cell overflow is possible.

24.4 Pictured Numeric Output

FORTH provides a formatting mechanism called pictured numeric output for constructing number strings character by character:

```

1  : HEX-PRINT  ( n -- )
2      BASE @ >R  HEX
3      <# # # # # # # # # #> TYPE CR
4      R> BASE ! ;

```

The sequence <# ... #> builds the number string in a scratch buffer. # converts one digit; #S converts all remaining digits; HOLD inserts a literal character; #> finishes and returns the string address and length.

A practical example — printing a number with a comma separator:

```

1  : .COMMA  ( n -- )
2      <#
3      #S
4      HOLD 44      \ 44 = comma character
5      #S
6      #>
7      TYPE CR ;

```

Chapter 25

Vocabularies and the Dictionary

25.1 How the Dictionary Is Organized

The StarForth dictionary is a linked list of word entries. The newest word is at the head; **FIND** searches from head toward tail. This means a newly-defined word with the same name as an existing word shadows the older one. The older word is not deleted — it is simply unreachable through the normal search path until the newer definition is removed with **FORGET**.

```
1 ok> : HELLO ." Version 1" CR ;
2 ok> HELLO
3 Version 1
4 ok> : HELLO ." Version 2" CR ;
5 ok> HELLO
6 Version 2
7 ok> FORGET HELLO
8 ok> HELLO
9 Version 1
10 ok>
```

FORGET removes the named word and all words defined after it. Use it carefully — definitions made after the word you want to forget will also disappear.

25.2 VOCABULARY

A vocabulary is a named wordlist. StarForth supports multiple vocabularies, allowing you to organize related words and avoid name collisions.

```
1 VOCABULARY GRAPHICS
2 GRAPHICS DEFINITIONS
3
4 : DRAW-LINE ... ;
5 : DRAW-CIRCLE ... ;
6
7 FORTH DEFINITIONS
```

After **GRAPHICS DEFINITIONS**, new words go into the **GRAPHICS** vocabulary. **FORTH DEFINITIONS** returns to defining into the **FORTH** vocabulary.

To make a vocabulary searchable, name it in the search order. The **ALSO** word adds a vocabulary to the front of the search order:

```
1 ok> ALSO GRAPHICS
2 ok> DRAW-LINE \ now visible
```

25.3 SEARCH-WORDLIST

SEARCH-WORDLIST searches a specific wordlist by name and count, returning an execution token and flags if found, or zero if not found:

```
1  ok> S" DRAW-LINE" FORTH-WORDLIST SEARCH-WORDLIST
```

This is useful when you want to test for word existence programmatically before attempting to execute it.

Chapter 26

Working with Text and I/O

26.1 Reading User Input

ACCEPT reads a line from the terminal:

```
1 CREATE INPUT-BUF 80 ALLOT
2
3 : GET-LINE ( -- addr count )
4   INPUT-BUF 80 ACCEPT ;
5
6 : ECHO-LINE
7   ." Enter text: "
8   GET-LINE TYPE CR ;
```

```
1 ok> ECHO-LINE
2 Enter text: Hello, FORTH!
3 Hello, FORTH!
4 ok>
```

KEY reads a single character without echoing:

```
1 : WAIT-FOR-ENTER
2   ." Press Enter to continue..."
3   BEGIN KEY 13 = UNTIL CR ;
```

26.2 Parsing Input

PARSE scans the input stream up to a delimiter and returns address and count. WORD scans to the next whitespace-delimited token and stores it as a counted string:

```
1 \ Parse everything up to the end of line
2 : PARSE-LINE ( -- addr count )
3   10 PARSE ; \ 10 = newline
```

WORD is used internally by the interpreter to tokenize input. At the prompt, you rarely need to call it directly.

26.3 Number Conversion

NUMBER (or NUMBER? in some implementations) attempts to parse a string as a number. In Star-Forth, the outer interpreter does this automatically when a word is not found in the dictionary. For programmatic parsing, use the base-aware conversion words.

26.4 Formatted Output

Build formatted output using the print words together:

```

1 : PRINT-ROW ( value label-addr label-len -- )
2   TYPE          \ print label
3   ." : "
4   .R ;          \ print value right-justified (needs width on
   stack)
5
6 : SHOW-STATS
7   ." --- Statistics ---" CR
8   42 S" Iterations" 7 ROLL .R CR
9   100 S" Percent" 7 ROLL .R CR ;

```

For tables, use .R (right-justified print) to align columns:

```

1 : PRINT-TABLE
2   ." N      N^2    N^3" CR
3   10 1 DO
4     I      5 .R
5     I I * 7 .R
6     I I * I * 9 .R
7     CR
8   LOOP ;

```

```

1 ok> PRINT-TABLE
2   N      N^2    N^3
3   1      1      1
4   2      4      8
5   3      9     27
6   4     16     64
7   5     25    125
8   6     36    216
9   7     49    343
10  8     64    512
11  9     81    729
12 ok>

```

Chapter 27

Writing and Loading Source Files

27.1 The .4th File Convention

StarForth source files use the .4th extension by convention. A .4th file is plain text containing FORTH words and definitions, exactly as you would type them at the prompt:

```
1 \ mylib.4th
2 \ A small library of utility words
3
4 : MAX3 ( a b c -- max )
5   ROT MAX MAX ;
6
7 : MIN3 ( a b c -- min )
8   ROT MIN MIN ;
9
10 : CLAMP ( n lo hi -- n' )
11   ROT \ ( lo hi n )
12   MIN \ ( lo n )
13   MAX ; \ ( result )
```

Load it with INCLUDE:

```
1 ok> S" mylib.4th" INCLUDE
2 ok> 5 1 10 CLAMP .
3 5 ok>
4 ok> 0 1 10 CLAMP .
5 1 ok>
6 ok> 99 1 10 CLAMP .
7 10 ok>
```

27.2 Comments in Source Files

FORTH supports two comment forms:

```
1 \ This is a line comment (backslash-space)
2 ( This is a paren comment )
```

The backslash comment extends to end of line. The paren comment extends to the closing parenthesis. Both are legal anywhere — in the REPL, in definitions, or in source files. By convention, stack effects use paren comments and explanations use backslash comments:

```
1 : HYPOTENUSE ( a b -- c )
2   \ computes sqrt(a^2 + b^2) using integer approximation
3   DUP * SWAP DUP * + ; ( a^2+b^2 -- needs SQRT )
```

27.3 Conditional Compilation

Use [IF] / [ELSE] / [THEN] for compile-time conditionals. These are immediate words that run at compile time rather than being compiled:

```
1 \ Define a debug-print word only in debug builds
2 HAS-DEBUG [IF]
3   : DPRINT ( n -- ) ." DEBUG: " . CR ;
4 [ELSE]
5   : DPRINT ( n -- ) DROP ; \ no-op in production
6 [THEN]
```

27.4 Loading Capsule Files

StarForth extends the standard FORTH file-loading mechanism with the EXEC word, which loads named capsule files by name from the capsule directory:

```
1 S"␣ACL.4th" EXEC \ load the ACL subsystem
2 S"␣doe.4th" EXEC \ load the DoE experiment harness
```

Capsules are content-addressed: the capsule ID is its XXHash64 content hash. Any modification to the capsule changes its hash, and the system detects the change at load time.

Chapter 28

The Test Suite

28.1 Running the Tests

StarForth ships 936+ tests covering every word category:

```
1 make test
```

Tests are organized in POST (Power-On Self Test) order:

1. Unit tests: Q48.16 fixed-point, inference statistics, decay slope inference.
2. Dictionary tests: FORTH-79 word validation across 22 categories.
3. Integration tests and stress tests.
4. Adversarial and fuzzing tests (`break_me_tests.c`).

Additional options:

```
1 make test FAIL_FAST=1      # Stop on first failure
2 make test PROFILE=1        # With basic profiling
```

28.2 Writing Your Own Tests

A minimal test pattern in FORTH is:

```
1 : TEST-EQUAL ( actual expected -- )
2   = IF ." PASS" ELSE ." FAIL" THEN CR ;
3
4 \ Test SQUARE
5 9 SQUARE 81 TEST-EQUAL
6 3 SQUARE 9 TEST-EQUAL
7 0 SQUARE 0 TEST-EQUAL
```

For larger test suites, use a counter:

```
1 VARIABLE TEST-PASS
2 VARIABLE TEST-FAIL
3 0 TEST-PASS !
4 0 TEST-FAIL !
5
6 : CHECK ( actual expected "desc" -- )
7   PARSE-LINE TYPE ." : "
8   = IF
9     ." PASS"
```

```
10      TEST-PASS @ 1 + TEST-PASS !
11  ELSE
12      ." FAIL "
13      TEST-FAIL @ 1 + TEST-FAIL !
14  THEN CR ;
15
16  : REPORT
17      ." Passed: " TEST-PASS @ . CR
18      ." Failed: " TEST-FAIL @ . CR ;
19
20  \ Usage:
21  9 SQUARE 81 CHECK square of 9
22  0 SQUARE 0 CHECK square of 0
23  REPORT
```

Part X

Advanced Topics

Chapter 29

The StarForth VM Internals for Users

You do not need to understand the VM implementation to use StarForth productively. But knowing a little about what happens under the hood explains several things you will observe in practice: why some words are faster than others, what the physics diagnostics are measuring, and how the adaptive runtime relates to the code you write.

29.1 How the Interpreter Loop Works

When you type a line at the `ok>` prompt, StarForth:

1. Tokenizes the input, splitting on whitespace.
2. For each token, searches the dictionary (hot-words cache first, then linear scan from the head of the dictionary linked list).
3. If found and in interpretation mode: executes the word immediately.
4. If found and in compilation mode (inside `:` ...): compiles a reference to the word into the current definition (unless the word is `IMMEDIATE`, in which case it executes at compile time).
5. If not found: attempts to parse the token as a number and push it.
6. If not a number: reports an error and aborts the current line.

Execution heat (`Loop #1`) is updated on every word execution. The rolling window (`Loop #2`) records each execution for statistical analysis. Both happen transparently.

29.2 Direct Threading

In the `fastest` build, StarForth uses direct threading: each compiled word body is a sequence of code pointers, and the inner interpreter jumps from one to the next using computed-goto. This eliminates an indirection level compared to indirect threading and accounts for a significant portion of the performance difference between the `debug` and `fastest` builds.

From a user's perspective, direct threading is invisible — words behave identically. The only observable effect is speed.

29.3 The Hot-Words Cache

StarForth maintains a hot-words cache: the most frequently executed words are sorted to the front of the dictionary linked list. Dictionary search always starts at the head, so hot words are found faster.

When you define a new word, the cache may be invalidated and rebuilt. For short interactive sessions this is imperceptible. For long-running programs that define many words at startup, you may notice a brief pause as the cache rebuilds — this is **Loop #1** doing its work.

WORD-ENTROPY shows the current heat values. Words you use heavily will show higher heat values; words you have not called recently will show lower values (heat decays over time via **Loop #3**).

29.4 Execution Tokens

An execution token (XT) is an opaque value that identifies a word. Tick (') produces an XT:

```
1 ok> ' DUP .
2 12345 ok> \ some dictionary address
```

The number printed is implementation-dependent — it is a VM address, not meaningful on its own. What you can do with an XT is pass it to **EXECUTE** to run the word later:

```
1 ok> ' DUP EXECUTE \ same as just typing DUP
```

Or store it for a callback:

```
1 VARIABLE HOOK
2 ' NOOP HOOK ! \ default: do nothing
3
4 : SET-HOOK ( xt -- ) HOOK ! ;
5 : RUN-HOOK ( -- ) HOOK @ EXECUTE ;
```

XTs are also the key for ACL lookups (Chapter 20).

Chapter 30

Performance Tuning Your Code

30.1 Measuring Before Optimizing

Use `WORD-ENTROPY` to identify which words are hot before attempting to optimize. Optimizing a cold word wastes effort.

The profiler gives more detail. Enable it at the shell level:

```
1 ./build/amd64/standard/starforth --profile 1 --profile-report <
   myprogram.4th
```

The report shows each word ranked by execution count. Typically 80–95% of all executions concentrate in the top ten words.

30.2 Build Profiles and Their Effect

The choice of build profile has a larger effect than any FORTH-level optimization for most programs:

Build	Relative speed	When to use
<code>debug</code>	1× (baseline)	Debugging
<code>all</code>	3–4×	Development testing
<code>fast</code>	5–6×	Normal production
<code>fastest</code>	6–7×	Benchmarks, deployment
<code>pgo</code>	7–8×	Maximum throughput

For most interactive use, `all` or `fast` is fine. For benchmarking or deployment, use `fastest` or `pgo`.

30.3 Physics-Aware Programming

The adaptive runtime favors frequently-used words. If you have a word that is called millions of times in a tight loop, you can expect it to migrate to the hot-words cache quickly. Conversely, setup words called only at startup will cool off and be pushed toward the tail of the dictionary list.

For reproducible benchmarks:

```
1 : BENCH-RUN ( iterations -- )
2   PHYSICS-RESET-STATS \ clear heat history
3   PHYSICS-FREEZE      \ stop adaptive updates
4   0 DO
```

```

5      MY-WORD-UNDER-TEST
6      LOOP
7      PHYSICS-THAW ;

```

PHYSICS-FREEZE ensures the cache and heat values do not change during measurement. PHYSICS-THAW restores normal operation afterward.

30.4 Stack Manipulation Cost

Stack operations (DUP, SWAP, ROT, etc.) are among the fastest operations in StarForth — they are direct array accesses on the stack. Excessive stack manipulation is a sign of design issues, not a performance problem per se. If you find yourself writing long sequences of ROT, SWAP, and OVER to get values in the right positions, consider using named variables instead:

```

1  \ Tangled stack version
2  : COMPLEX ( a b c d -- result )
3      ROT ROT OVER * ROT ROT ... ;
4
5  \ Cleaner with locals / variables
6  VARIABLE _A VARIABLE _B VARIABLE _C VARIABLE _D
7
8  : COMPLEX ( a b c d -- result )
9      _D ! _C ! _B ! _A !
10     _A @ _B @ * _C @ _D @ * + ;

```

The variable version is slightly slower per operation but far easier to read and debug.

Chapter 31

The Block Editor

31.1 Blocks as Source Storage

Traditional FORTH stored all source code in 1024-byte blocks on disk. StarForth preserves this model. A block holds up to 16 lines of 64 characters each, no newlines — a fixed-format screen of text.

To list the contents of a block:

```
1 ok> 2048 LIST
```

This prints the 16 lines of block 2048, line-numbered. Empty blocks print as blank lines.

To load (execute) a block:

```
1 ok> 2048 LOAD
```

To load a range of blocks:

```
1 ok> 2048 2055 THRU
```

31.2 Block Storage Backends

StarForth supports three block storage backends:

RAM Default for interactive sessions. Blocks live in VM memory. Fast but not persistent across sessions.

File Blocks are stored in a file on the host filesystem. Persistent. Configured by `STARFORTH_BLOCK_FILE` environment variable.

Factory Read-only blocks baked into the binary. Used for capsule content in the bare-metal kernel.

On a standard hosted build, the default is RAM. To use file-backed blocks:

```
1 STARFORTH_BLOCK_FILE=myblocks.blk ./build/amd64/standard/starforth
```

31.3 Protecting Blocks with FLUSH

If you have written to file-backed blocks and want to ensure changes are saved before exiting:

```
1 ok> FLUSH
2 ok> BYE
```

FLUSH writes all modified (dirty) block buffers back to the backing store. Exiting without **FLUSH** may lose changes if the backend does not auto-flush on close.

Chapter 32

Capsules and the Boot System

32.1 What a Capsule Is

A capsule is a self-contained FORTH source file whose identity is its content hash. The capsule ID is the XXHash64 of the capsule’s contents — any change to the file produces a different ID. This makes capsules tamper-evident: load a capsule, compute its hash, compare to the expected ID, and you know whether the content is unchanged.

Capsule files live in the `capsules/` directory and use the `.4th` extension.

32.2 `init.4th`: The Default Personality

When StarForth starts, it loads `init.4th` from `./conf/` (on a hosted build). This file defines the foundational word set, installs a dictionary fence, and zeros blocks for user use.

The fence means that words defined in `init.4th` cannot be removed with `FORGET`. This protects the core vocabulary from accidental deletion.

32.3 Capsule Variants

StarForth ships several `init.4th` variants for different operating modes:

File	Purpose
<code>init.4th</code>	Default Mama VM personality
<code>init-0.4th</code> through <code>init-9.4th</code>	Numbered personality variants
<code>init-l8-stable.4th</code>	L8 Jacquard stable mode
<code>init-l8-volatile.4th</code>	L8 Jacquard volatile mode
<code>init-l8-diverse.4th</code>	L8 Jacquard diverse mode
<code>init-l8-temporal.4th</code>	L8 Jacquard temporal mode
<code>init-l8-transition.4th</code>	L8 Jacquard transition mode
<code>init-l8-omni.4th</code>	L8 Jacquard omni mode
<code>ACL.4th</code>	Word-level ACL subsystem (opt-in)
<code>zuse.4th</code>	Bootstrap superuser (loaded by <code>ACL.4th</code>)

The L8 Jacquard variants configure the Steady-State Machine (`ssm_jacquard.c`) to operate in a specific mode. Each mode biases the adaptive runtime toward different behavior: stable prefers low-variance execution, diverse favors broad word coverage, temporal weights recent history more heavily, and so on. These are intended for DoE experimentation rather than everyday use.

32.4 The Birth Protocol

In the bare-metal kernel (LithosAnanke), StarForth VMs are born through a birth protocol:

1. Locate the capsule by name.
2. Validate the content hash.
3. Allocate a VM ID.
4. Execute the capsule's IDENTITY block (the capsule code).
5. Execute the PERSONALITY block (block 1 from the ramdrive).
6. Log a parity record: VM ID + capsule hash + dictionary hash.

The parity record enables offline determinism verification: an independent system can load the same capsule and compare dictionary hashes. Identical hashes mean identical dictionaries — the definition of 0% algorithmic variance.

For hosted interactive use, the birth protocol runs transparently when StarForth starts. You only interact with its results (the loaded dictionary), not the protocol itself.

Chapter 33

The L8 Jacquard Mode Selector

33.1 What the Jacquard Selector Does

The L8 Jacquard Mode Selector is an additional layer above the seven feedback loops. It monitors the distribution of execution patterns across attractor buckets (clusters of similar execution histories) and switches between six predefined mode configurations based on which attractor the system is approaching.

Named after Joseph Marie Jacquard, whose programmable loom used punch cards to control complex patterns, the Jacquard Selector treats the adaptive runtime as a programmable pattern — selecting different behavior profiles depending on the observed workload dynamics.

33.2 The Six Modes

stable Favors low-variance, predictable execution. Decay slopes are conservative; the hot-words cache changes slowly. Best for long-running workloads with stable access patterns.

volatile Responds quickly to changes in access patterns. Decay is aggressive; the cache refreshes rapidly. Best for workloads with rapidly shifting hot paths.

diverse Biases toward broad dictionary coverage. Prevents a small set of words from monopolizing the cache at the expense of moderately-used words. Best for exploratory interactive sessions.

temporal Weights recent execution history more heavily. The rolling window shrinks to focus on the most recent activity. Best for workloads with clear phases.

transition Intermediate mode used during mode changes. Smooths the transition between attractor basins.

omni All attractor statistics are weighted equally. No single mode dominates. Used when the workload does not fit a single characterization.

33.3 Mode Selection for Users

In everyday interactive use you will not configure the Jacquard Selector directly. It operates automatically based on workload statistics. The L8 mode variants of `init.4th` (`init-l8-stable.4th`, etc.) pre-configure the Selector to start in a specific mode for DoE experiments where you want to observe behavior in a controlled attractor.

If you are running a long benchmark and want to observe which mode the system is in, use `WORD-ENTROPY` — the output includes SSM state information in builds where that diagnostic is enabled.

Chapter 34

Formal Verification

34.1 What Has Been Proved

StarForth ships 24 Isabelle/HOL theory files providing machine-checkable proofs of correctness and determinism. These proofs live in the `proof/` directory and cover:

- All seven feedback loops (Loop #1 through Loop #7), proving that each accumulation–stabilization pair behaves as designed.
- Core word categories: arithmetic, stack operations, logical operations, memory operations, return-stack operations, and Q48.16 fixed-point.
- Overall correctness (`StarForth_Correctness.thy`), concurrency properties (`StarForth_Concurrent`), state transitions, and mutual exclusion.
- ACL system (five additional theories): pin monotonicity, inheritance clearing pin, TTL boundedness, emergency bypass, and no-escalation.

34.2 What Determinism Means Here

The proofs establish that the adaptive runtime is *algorithmically deterministic*: given the same sequence of word executions and the same initial state, the runtime produces the same internal state every time. This property was validated experimentally across 90 runs with 0% variance.

Algorithmic determinism does not mean the same wall-clock times — timing depends on the host hardware, OS scheduler, and other factors outside StarForth’s control. It means the logic of adaptation (which words are promoted to the cache, what decay slope is computed, what window size is selected) is fully determined by the execution history, not by any random or non-deterministic process.

This matters for reproducible deployments: two instances of StarForth loading the same capsule and receiving the same inputs will arrive at the same dictionary heat distribution.

34.3 Checking the Proofs

To check the Isabelle/HOL proofs (requires an Isabelle installation):

```
1 isabelle build -D proof/
```

See `docs/01-getting-started/DEVELOPER.md` for Isabelle installation instructions and compatible versions.

Chapter 35

StarForth on L4Re and Embedded Targets

35.1 L4Re Microkernel

StarForth can run on the L4Re microkernel operating system. The L4Re build target enables the L4Re block-storage backend and ROMFS file access in place of POSIX:

```
1 make l4re
```

The L4Re build implies `STARFORTH_MINIMAL=1`: ANSI color codes are suppressed and I/O is reduced to minimal output. The build links without the standard C library (`-nostdlib -ffreestanding`).

Status: partially implemented. The ROMFS stub is present; full L4Re integration is ongoing.

35.2 Minimal / Freestanding Builds

The minimal build produces a freestanding binary with no libc dependencies:

```
1 make minimal
```

This is useful for embedded environments or as a starting point for porting to a new operating system. The minimal build disables features that depend on POSIX or the C library. You can add features back selectively by enabling the corresponding build flags.

35.3 Cross-Compilation

Cross-compile for AArch64 from an x86_64 host:

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make CC=aarch64-linux-gnu-gcc rpi4-cross
3 scp build/starforth user@target-device:~/
```

The cross-compiled binary is statically linked and requires no runtime dependencies on the target. Verify the architecture:

```
1 file build/starforth
2 # ELF 64-bit LSB executable, ARM aarch64, statically linked
```

Cross-compilation for RISC-V 64 follows the same pattern with the appropriate cross-toolchain (`gcc-riscv64-linux-gnu`).

Part XI

Recipes

Chapter 36

Frequently Used Idioms

This chapter collects short FORTH idioms you will reach for repeatedly. Each entry shows the idiom, its stack effect, and a concrete example. They are organized by what you are trying to accomplish, not by word name.

36.1 Swap and Drop a Pair

Discard the second item from the top:

```
1  ( a b -- b )
2  SWAP DROP          \ explicit
3  NIP                \ compact equivalent
```

Use NIP when you compute something from two values but only need the result of the second operation:

```
1  : AREA-FROM-SIDES ( width height -- area )
2    * ;           \ stack: width height -- area; no NIP needed here
```

```
1  : LAST-OF-THREE ( a b c -- c )
2    NIP NIP ;
```

36.2 Apply an Operation to Two Copies

TUCK lets you use a value in two consecutive operations without re-fetching it from a variable:

```
1  ( n -- n*n n )      \ square, then original
2  TUCK *
```

```
1  ( a b -- b a b )
2  TUCK
```

Use TUCK when you need the top value both as an operand and to keep it on the stack:

```
1  : PERCENT-OF ( total pct -- value remaining )
2    \ pct of total, then total minus that value
3    TUCK 100 */ DUP ROT SWAP - ;
```

36.3 Conditional Increment

Increment a counter only if a condition is true. FORTH's flag convention (-1 for true, 0 for false) means you can add a flag directly:

```

1 ( counter condition -- counter+1/counter )
2 +      \ adds -1 (true) or 0 (false) -- this subtracts 1!
3 SWAP -  \ correct: condition is -1, negating gives 1

```

More clearly:

```

1 : COND-INC ( n flag -- n' )
2   IF 1 + THEN ;

```

```

1 ok> 5 1 3 > COND-INC .      \ 1 > 3 is false (0), so no increment
2 5 ok>
3 ok> 5 3 1 > COND-INC .      \ 3 > 1 is true (-1), so increment
4 6 ok>

```

36.4 Clamp a Value to a Range

```

1 : CLAMP ( n lo hi -- n' )
2   ROT      \ ( lo hi n )
3   MIN      \ ( lo min(hi,n) )
4   MAX ;    \ ( max(lo, min(hi,n)) )

```

```

1 ok> 15 0 10 CLAMP .
2 10 ok>
3 ok> -5 0 10 CLAMP .
4 0 ok>
5 ok> 7 0 10 CLAMP .
6 7 ok>

```

36.5 Absolute Difference

```

1 : ABS-DIFF ( a b -- |a-b| )
2   - ABS ;

```

36.6 Integer Division with Rounding

FORTH's / truncates toward zero. For rounded division:

```

1 : ROUND-DIV ( a b -- a/b rounded )
2   2DUP /      \ quotient
3   -ROT MOD     \ remainder ( quot rem )
4   SWAP OVER    \ ( rem quot rem )
5   2/ ABS > IF  \ remainder > |divisor/2|?
6     1 +        \ round up
7   THEN ;

```

Or, more simply for positive values:

```

1 : ROUND-DIV+ ( a b -- rounded )
2   2DUP 2/ + SWAP / ; \ (a + b/2) / b

```

36.7 Swap Top Two Booleans

Because FORTH booleans are plain integers, boolean operations are just arithmetic. NOT inverts, AND is bitwise AND (works correctly on -1/0 flags), and OR is bitwise OR:

```

1 ok> -1 0 AND .      \ true AND false = false
2 0 ok>
3 ok> -1 0 OR .       \ true OR false = true
4 -1 ok>
5 ok> -1 NOT .        \ NOT true = false
6 0 ok>

```

36.8 String Equality Test

Compare two strings of known length:

```

1 : STR= ( addr1 len1 addr2 len2 -- flag )
2   ROT OVER <> IF 2DROP DROP 0 EXIT THEN
3   \ lengths are equal; now compare bytes
4   OVER SWAP ( addr1 addr2 len )
5   0 DO
6     OVER C@ OVER C@ <> IF
7       2DROP 0 UNLOOP EXIT
8     THEN
9     SWAP 1+ SWAP 1+
10  LOOP
11  2DROP -1 ;

```

36.9 Fibonacci Sequence

Classic Fibonacci using the return stack as a third register:

```

1 : FIB ( n -- fib(n) )
2   DUP 2 < IF EXIT THEN \ base cases: fib(0)=0, fib(1)=1
3   DUP 1 - RECURSE      \ fib(n-1)
4   SWAP 2 - RECURSE     \ fib(n-2)
5   + ;

```

```

1 ok> 10 FIB .
2 55 ok>

```

For larger values, use the iterative form (much faster):

```

1 : FIB-ITER ( n -- fib(n) )
2   DUP 2 < IF EXIT THEN
3   1 0 \ a=1, b=0 (fib(1), fib(0))
4   ROT 1 - 0 DO
5     OVER + \ new a = a+b
6     SWAP \ rotate
7   LOOP
8   DROP ;

```

```

1 ok> 40 FIB-ITER .
2 102334155 ok>

```

36.10 Printing a Separator Line

```
1 : HLINE ( n -- )
2   0 DO 45 EMIT LOOP CR ; \ 45 = '-'
3
4 : BANNER ( addr len -- )
5   DUP 2 + >R
6   ." +-" R@ HLINE ." | "
7   2DUP TYPE CR ." +-" R> HLINE ;
```

```
1 ok> S" StarForth v3.1" BANNER
2 +-----+
3 | StarForth v3.1
4 +-----+
5 ok>
```

Chapter 37

Working Examples

37.1 A Simple Stack Calculator

This example builds an interactive RPN calculator on top of the StarForth REPL. Because StarForth IS an RPN calculator, this is mostly a demonstration of how you can add user-level commands on top:

```
1  \ A FORTH-based RPN calculator session
2
3  \ Define helper display words
4  : SHOW-STACK .S CR ;
5  : RESULT      . CR ;
6
7  \ Factorial
8  : FACT ( n -- n! )
9      DUP 0= IF DROP 1 EXIT THEN
10     DUP 1 - RECURSE * ;
11
12 \ Power
13 : POW ( base exp -- base^exp )
14     1 SWAP 0 DO OVER * LOOP NIP ;
```

Using these at the REPL:

```
1  ok> 5 FACT RESULT
2  120
3  ok> 2 10 POW RESULT
4  1024
5  ok> 12 FACT RESULT
6  479001600
```

37.2 A Data Table

Store and retrieve structured data using CREATE and array indexing:

```
1  \ A table of (name-hash, value) pairs
2  \ 5 entries, 2 cells each
3
4  CREATE DATA-TABLE 10 CELLS ALLOT
5
6  : TABLE-SET ( value index -- )
7      2* CELLS DATA-TABLE + ! ;
8
9  : TABLE-GET ( index -- value )
```

```

10      2* CELLS DATA-TABLE + @ ;
11
12 \ Initialize
13 100  0 TABLE-SET
14 200  1 TABLE-SET
15 300  2 TABLE-SET
16 400  3 TABLE-SET
17 500  4 TABLE-SET
18
19 : DUMP-TABLE
20   ." Index  Value" CR
21   5 0 DO
22     I 4 .R I TABLE-GET 8 .R CR
23   LOOP ;

```

```

1 ok> DUMP-TABLE
2 Index  Value
3      0    100
4      1    200
5      2    300
6      3    400
7      4    500
8 ok>

```

37.3 A Simple Moving Average

Compute a moving average over the last N readings using a circular buffer:

```

1 10 CONSTANT MA-SIZE
2 CREATE MA-BUF MA-SIZE CELLS ALLOT
3 VARIABLE MA-INDEX
4 VARIABLE MA-SUM
5
6 0 MA-INDEX !
7 0 MA-SUM !
8 MA-BUF MA-SIZE CELLS 0 FILL \ zero the buffer
9
10 : MA-UPDATE ( new-value -- average )
11   \ subtract old value at current index
12   MA-INDEX @ CELLS MA-BUF + @
13   MA-SUM @ SWAP - MA-SUM !
14   \ store new value
15   DUP MA-INDEX @ CELLS MA-BUF + !
16   \ add to sum
17   MA-SUM @ + MA-SUM !
18   \ advance index circularly
19   MA-INDEX @ 1 + MA-SIZE MOD MA-INDEX !
20   \ return average
21   MA-SUM @ MA-SIZE / ;
22
23 : TEST-MA
24   ." Moving average test (window=10)" CR
25   20 0 DO
26     I 3 * \ input: 0,3,6,9,...,57
27     MA-UPDATE
28     ." Input: " I 3 * 4 .R
29     ." Avg: " . CR

```

```
30 | LOOP ;
```

37.4 A Minimal State Logger

Log timestamped events to a block buffer:

```
1 \ Minimal event logger using block 2048
2 2048 CONSTANT LOG-BLOCK
3 0 CONSTANT LOG-OFFSET \ offset within block (in bytes)
4 VARIABLE LOG-POS
5 0 LOG-POS !
6
7 : LOG-RESET
8   0 LOG-POS !
9   LOG-BLOCK BUFFER 1024 0 FILL UPDATE ;
10
11 : LOG-WRITE ( addr count -- )
12   LOG-POS @ 1024 < IF
13     LOG-BLOCK BUFFER LOG-POS @ +
14     SWAP MOVE
15     LOG-POS @ + LOG-POS !
16   ELSE
17     ." Log full" CR
18   THEN ;
19
20 : LOG-MSG ( addr count -- )
21   2DUP LOG-WRITE DROP DROP ;
22
23 : LOG-DUMP
24   LOG-BLOCK BLOCK LOG-POS @ TYPE CR ;
```

37.5 Physics-Aware Benchmarking

A benchmark harness that controls the adaptive runtime for reproducible measurements:

```
1 \ Benchmark framework using physics controls
2
3 VARIABLE BENCH-ITERATIONS
4 100000 BENCH-ITERATIONS !
5
6 : BENCH-WORD ( xt -- )
7   \ Warm up (let physics settle)
8   PHYSICS-RESET-STATS
9   1000 0 DO DUP EXECUTE LOOP
10  \ Now freeze and measure
11  PHYSICS-FREEZE
12  BENCH-ITERATIONS @ 0 DO DUP EXECUTE LOOP
13  PHYSICS-THAW
14  DROP ;
15
16 \ Example: benchmark DUP
17 : RUN-DUP-BENCH
18   ." Benchmarking DUP x "
19   BENCH-ITERATIONS @ . ." iterations" CR
20   ' DUP BENCH-WORD
21   ." Done." CR ;
```

The `PHYSICS-FREEZE/PHYSICS-THAW` pair ensures that heat updates, cache evictions, and decay events do not occur during the timed region, giving cleaner timing measurements.

Chapter 38

Quick Reference: Key Sequences

The StarForth REPL runs in a standard POSIX terminal. The following key sequences are handled by the terminal or readline layer:

Key sequence	Effect
Enter	Execute the current input line
Ctrl-C	Interrupt current operation; return to prompt
Ctrl-D	End of input (exits REPL, same as BYE)
Backspace	Delete previous character
Up arrow	Previous history entry
Down arrow	Next history entry
Ctrl-A	Move cursor to beginning of line
Ctrl-E	Move cursor to end of line
Ctrl-K	Kill to end of line
Ctrl-U	Kill entire line

Chapter 39

Migrating from Other FORTH Systems

39.1 From gforth or ANS FORTH

StarForth targets FORTH-79 compliance. It is broadly compatible with ANS FORTH (FORTH-94) and gforth programs, with a few differences to watch:

S" handling StarForth's S" behaves as in ANS FORTH: it pushes an address and count. Programs that assume counted-string format (a length byte followed by characters) need to use COUNT to convert.

Character size ANS FORTH specifies CHAR as one address unit. StarForth uses byte-addressed cells, so CHAR+ is 1+ and CHARS is a no-op. Code that uses CHARS for portability will work correctly.

Cell size StarForth cells are 64 bits on a 64-bit host. Programs that assume 32-bit cells will need adjustment if they use bit patterns that fill a cell exactly.

LOCAL variables StarForth does not implement ANS FORTH local variables (LOCALS| / TO). Use named VARIABLES instead, or restructure to keep values on the stack.

THROW/CATCH StarForth's exception handling may differ from ANS FORTH.

Floating-point StarForth does not implement the ANS FORTH floating-point word set. Use Q48.16 fixed-point (Q+, Q*, etc.) or integer arithmetic for fractional values.

39.2 From FORTH-79

StarForth is FORTH-79 compliant. Programs written for the original FORTH-79 standard should run without modification. The main additions StarForth makes beyond FORTH-79 are:

- The physics adaptive runtime (seven feedback loops), which is entirely transparent unless you call WORD-ENTROPY, PHYSICS-FREEZE, or PHYSICS-THAW.
- The ACL word-level access control system, disabled by default.
- Q48.16 fixed-point arithmetic words.
- The block subsystem with pluggable backends.
- StarForth-specific extensions in `starforth_words.c`.

None of these conflict with FORTH-79 code. They simply add capability on top.

39.3 From C or Python

The largest adjustment is postfix notation and the stack discipline. Some practical advice:

- Draw a picture of the stack before each operation while learning. Writing (`a b - c`) for every sub-expression helps.
- Use `.S` compulsively at first. It does not consume anything, so you can sprinkle it everywhere.
- Resist the urge to write long, complex word bodies. Five lines is often too many. Factor into smaller words.
- `VARIABLE` fills the role of a local variable but it is global within the VM — be careful with re-entrancy if a word using a `VARIABLE` calls itself or a word that also uses the same variable.
- There are no types. You decide what each stack cell means. A cell could be an integer, an address, a boolean, a character, or a $\mathbb{Q}_{48.16}$ value. Naming your stack-effect slots meaningfully (`( count addr - result )`) is your type system.

Part XII

Running on Hardware

Chapter 40

Linux and POSIX Platforms

StarForth's standard build requires only GCC (or Clang) and GNU Make. No additional runtime dependencies are needed beyond the standard C library.

```
1 make fastest
2 ./build/amd64/standard/starforth
```

The binary is statically linked on supported configurations. No shared libraries are needed on the target system.

Chapter 41

Raspberry Pi 4

41.1 Building StarForth on Raspberry Pi 4

This guide covers building and tuning StarForth for the Raspberry Pi 4 with its ARM64 assembly optimizations. The board pairs a Broadcom BCM2711 (quad-core Cortex-A72 at 1.5 GHz, ARMv8-A) with 32 KB of L1 instruction and data cache per core, 1 MB of shared L2, and LPDDR4-3200 memory; NEON and CRC32 are both present.

41.1.1 Building

The Pi 4 can be built natively or cross-compiled. A native build installs `build-essential`, clones the repository, and compiles with the Cortex-A72 tuning flags:

```
1 make CFLAGS="-std=c99 -O3 -march=armv8-a+crc+simd -mtune=cortex-a72 -\
2   -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -Iinclude -\
3   -Isrc/word_source -Isrc/test_runner/include"
```

Cross-compiling from x86_64 Linux installs `gcc-aarch64-linux-gnu`, builds with `CC=aarch64-linux-gnu-gcc` and `LDFLAGS="-static"`, and copies the static binary to the board over `scp`. The repository provides convenience targets: `rpi4` for a native build, `rpi4-cross` for a static cross-build, and `rpi4-perf` for the maximum-performance build with direct threading and LTO.

41.1.2 Architecture Detection and Code Integration

A detection header (`include/arch_detect.h`) selects the right optimization headers and primitive macros at compile time — `x86_64`, `AArch64`, or a portable fallback with a warning. Stack management, arithmetic words, and dictionary lookup each keep an assembly path guarded by `USE_ASM_OPT` alongside a C path, so the architecture-agnostic `vm_push_opt/vm_pop_opt` macros resolve to the best available implementation.

41.1.3 Performance Tuning

Three techniques dominate on the Cortex-A72. Aligning the `VM` struct to the 64-byte cache line and grouping hot data (stacks and pointers) away from cold data reduces false sharing. Prefetching the next dictionary entry while comparing the current one hides traversal latency. And NEON block copy moves 64 bytes per iteration for bulk memory operations, falling back to scalar copy for the remainder.

41.1.4 Benchmarking

Reliable measurement requires pinning the CPU to performance mode and watching temperature. The governor is set with:

```
1 echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/
  scaling_governor
2 watch -n 1 vcgencmd measure_temp
3 vcgencmd get_throttled          # 0x0 means no throttling occurred
```

A benchmark script builds an -O2 baseline and the optimized `rpi4-perf` binary, runs a million iterations each of stack, arithmetic, and logic loops, and reports the timing difference along with final temperature and throttling status. The `perf` tool supplies cache-reference, cache-miss, branch, and branch-miss counters for deeper analysis.

41.1.5 Thermal and Power

Sustained workloads need cooling. A passive aluminum or copper heatsink covering the entire CPU with a 1 mm thermal pad is the minimum; a 5 V 30 mm fan, optionally GPIO-controlled, handles continuous load. Power draw runs 2–3 W idle, 4–5 W under light load, and 7–8 W with all cores busy, plus 2–3 W for peripherals — the official 5 V/3 A supply is recommended. Overclocking through `/boot/config.txt` is possible but requires adequate cooling and may void the warranty.

41.1.6 Storage and Deployment

For the SD card, choose UHS-I or better with an A2 application-performance rating; brands with strong random I/O include Samsung EVO Plus, SanDisk Extreme, and Kingston Canvas React. For production, a systemd service unit runs StarForth on boot with automatic restart on failure; boot time can be trimmed by disabling unused features and the splash screen in `/boot/config.txt`; and standard hardening (system updates, a UFW firewall, and key-only SSH) rounds out a deployment.

41.1.7 Troubleshooting

- *Illegal instruction at runtime* — the build exceeded the host’s feature set; rebuild with the conservative `-march=armv8-a`.
- *Cross-compiled binary will not run* — confirm it is statically linked (`file build/starforth`); rebuild with `-static` otherwise.
- *Slower than expected* — verify the governor is `performance` rather than `ondemand` and check `vcgencmd get_throttled` for thermal throttling.
- *Out of memory* — reduce `VM_MEMORY_SIZE` or add swap.

Chapter 42

LithosAnanke Bare-Metal Boot

LithosAnanke (“stone inevitability”) is the bare-metal UEFI kernel that boots StarForth directly on hardware without a host operating system. Version 1.5.3 ships as a monolithic UEFI PE32+ executable.

42.1 Boot Sequence

```
1  UEFI Firmware -> uefi_loader.c (BOOTX64.EFI)
2      ExitBootServices() -- kernel owns hardware
3      -> kernel_main()
4          M1: Console (UART 16550 + framebuffer)
5          M2: Physical memory manager
6          M3: Virtual memory (4-level paging)
7          M4: IDT + APIC interrupts
8          M5: TSC + HPET + APIC timer (100 Hz heartbeat)
9          M6: kmalloc heap
10         M7: StarForth VM + capsule loading -> ok> REPL
```

Milestones M0–M6 are complete. M7 (StarForth VM integration) is in progress.

42.2 Building the Kernel

```
1  # AMD64
2  make -f Makefile.starkernel
3
4  # AArch64
5  make -f Makefile.starkernel ARCH=aarch64
6
7  # RISC-V 64
8  make -f Makefile.starkernel ARCH=riscv64
9
10 # Run in QEMU with OVMF
11 make -f Makefile.starkernel qemu
```

Output: build/amd64/kernel/starkernel_loader.efi (UEFI PE32+ executable).

Appendix A

Build Flags Reference

A.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

A.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: $\approx 20\text{--}25\%$ on `x86_64`; $\approx 15\text{--}20\%$ on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: $\approx 30\text{--}40\%$ over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

NDEBUG

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: $\approx 5\text{--}10\%$ speedup.

STARFORTH_PERFORMANCE

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: $\approx 5\text{--}8\%$ over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

A.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

ARCH_X86_64

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

ARCH_ARM64

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

ARCH_NAME

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

A.1.3 Platform / Target Options

STARFORTH_MINIMAL

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make 14re`.

L4RE_TARGET

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

A.1.4 Debugging and Profiling Options**DEBUG**

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

PROFILE_ENABLED

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: $\approx 10\text{--}15\%$.

```
1 ./build/starforth --profile 2          # enable at depth 2
2 ./build/starforth --profile-report # print results
```

See `include/profiler.h`.

STRICT_PTR

Controls the Forth address-to-C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

A.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

A.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40–42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

A.1.7 Feature Flags

STARFORTH_ANSI

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

VM_HAS_CURRENT_ENTRY

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

A.1.8 Build System Variables

- **CC** — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- **CFLAGS** — compiler flags. Extends Makefile `BASE_CFLAGS`.
- **LDFLAGS** — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- **MINIMAL** — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- **ASM** — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

A.1.9 Build Configuration Matrix

Table A.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
<code>debug</code>	0	0	No	<code>-O0 -g</code>	Development
<code>all</code>	1	0	No	<code>-O2</code>	Default build
<code>turbo</code>	1	0	Yes	<code>-O3 -flto</code>	Fast, no debug
<code>fast</code>	1	1	Yes	<code>-O3</code>	Fast, readable
<code>fastest</code>	1	1	Yes	<code>-O3 -flto</code>	Maximum speed
<code>pgo</code>	1	1	Yes	<code>-O3 -fprofile-use</code>	Absolute fastest
<code>minimal</code>	0	0	No	<code>-O2</code>	Embedded
<code>l4re</code>	0	0	No	<code>-O2</code>	L4Re microkernel
<code>profile</code>	0	0	No	<code>-O1 -g</code>	Profiling

A.1.10 Optimization Level Guidelines

- **-O0 -g (development)** — fastest compile, slowest runtime ($\approx 10\times$ below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **-O2 (balanced)** — moderate compile overhead; $3\text{--}4\times$ runtime gain over `-O0`. Default production-test builds.
- **-O3 (high performance)** — aggressive inlining and vectorization; $5\text{--}6\times$ over `-O0`. Production releases.

- **-O3 -flto -march=native -fprofile-use (maximum)** — multi-stage build; 7–8× over -O0. Release and benchmark artifacts. Use `make pgo`.

A.1.11 Usage Examples

Maximum performance native build

```
1 make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
1 make clean && make profile
2 ./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
1 make rpi4-cross
2 scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
1 make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
1 make clean
2 make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
3     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
4     LDFLAGS="-flto -s"
```

Generate assembly listings

```
1 make asm
2 less build/stack_management.s
```

A.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

A.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: $\approx 7\text{--}8\times$ faster than a debug build; $\approx 1.2\text{--}1.5\times$ faster than a plain `-O3` build.

A.1.14 Troubleshooting

“Unknown architecture” warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is `x86_64` or `ARM64`, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

Appendix B

FORTH-79 Word Quick Reference

This table covers the full FORTH-79 word set available in StarForth, organized alphabetically within categories. Stack effects use standard notation: (**before** - **after**), where items are listed bottom-to-top and the rightmost item is the top of stack.

Flags are -1 (true) or 0 (false) unless noted otherwise. Addresses (**addr**) are VM byte offsets, not C pointers.

Stack Words

Word	Stack Effect	Description
DUP	(n - n n)	Duplicate top item
DROP	(n -)	Discard top item
SWAP	(a b - b a)	Exchange top two items
ROT	(a b c - b c a)	Rotate top three; third comes to top
OVER	(a b - a b a)	Copy second item to top
NIP	(a b - b)	Drop second item
TUCK	(a b - b a b)	Copy top item below second
2DUP	(a b - a b a b)	Duplicate top pair
2DROP	(a b -)	Drop top pair
2SWAP	(a b c d - c d a b)	Exchange top two pairs
2OVER	(a b c d - a b c d a b)	Copy second pair to top
DEPTH	(- n)	Number of items on the data stack
PICK	(xn...x0 n - xn...x0 xn)	Copy nth item to top
ROLL	(xn...x0 n - xn-1...x0 xn)	Rotate nth item to top

Return Stack Words

Word	Stack Effect	Description
>R	(n -) (R: - n)	Move top data item to return stack
R>	(- n) (R: n -)	Move top return item to data stack
R@	(- n) (R: n - n)	Copy top of return stack to data stack
RDROP	(-) (R: n -)	Discard top of return stack
2>R	(a b -) (R: - a b)	Move top data pair to return stack
2R>	(- a b) (R: a b -)	Move top return pair to data stack
2R@	(- a b) (R: a b - a b)	Copy top return pair to data stack

Arithmetic Words

Word	Stack Effect	Description
+	(a b – a+b)	Integer addition
-	(a b – a-b)	Integer subtraction
*	(a b – a*b)	Integer multiplication
/	(a b – a/b)	Integer division (truncate toward zero)
MOD	(a b – a mod b)	Integer remainder
/MOD	(a b – rem quot)	Division and remainder
ABS	(n – n)	Absolute value
NEGATE	(n – -n)	Arithmetic negation
MIN	(a b – min)	Minimum of two values
MAX	(a b – max)	Maximum of two values
1+	(n – n+1)	Add 1
1-	(n – n-1)	Subtract 1
2+	(n – n+2)	Add 2
2-	(n – n-2)	Subtract 2
2*	(n – n*2)	Multiply by 2 (left shift 1)
2/	(n – n/2)	Divide by 2 (arithmetic right shift 1)

Double-Cell Arithmetic

Word	Stack Effect	Description
D+	(d1 d2 – d1+d2)	Double-cell addition
D-	(d1 d2 – d1-d2)	Double-cell subtraction
D=	(d1 d2 – flag)	Double-cell equality
D<	(d1 d2 – flag)	Double-cell signed less-than
DNEGATE	(d – -d)	Double-cell negation
DABS	(d – d)	Double-cell absolute value
M+	(d n – d+n)	Add single to double
M*	(a b – d)	Multiply; produce double-cell result
UM*	(u1 u2 – ud)	Unsigned multiply; produce double-cell result

Logical and Bitwise Words

Word	Stack Effect	Description
AND	(a b – a&b)	Bitwise AND (also logical AND on flags)
OR	(a b – a b)	Bitwise OR (also logical OR on flags)
XOR	(a b – a xor b)	Bitwise exclusive OR
INVERT	(n – ~n)	Bitwise complement (all bits flipped)
NOT	(n – flag)	Logical NOT: -1 if n=0, else 0
LSHIFT	(n count – n')	Left shift n by count bits
RSHIFT	(n count – n')	Logical right shift n by count bits

Comparison Words

Word	Stack Effect	Description
=	(a b – flag)	Equal
<>	(a b – flag)	Not equal
<	(a b – flag)	Signed less-than
>	(a b – flag)	Signed greater-than
<=	(a b – flag)	Signed less-than-or-equal
>=	(a b – flag)	Signed greater-than-or-equal
U<	(u1 u2 – flag)	Unsigned less-than
U>	(u1 u2 – flag)	Unsigned greater-than
0=	(n – flag)	True if n equals zero
0<>	(n – flag)	True if n is non-zero
0<	(n – flag)	True if n is negative
0>	(n – flag)	True if n is positive

Memory Words

Word	Stack Effect	Description
@	(addr – n)	Fetch one cell from addr
!	(n addr –)	Store one cell at addr
C@	(addr – char)	Fetch one byte from addr
C!	(char addr –)	Store one byte at addr
2@	(addr – d)	Fetch double-cell (two consecutive cells)
2!	(d addr –)	Store double-cell
MOVE	(src dst count –)	Copy count bytes from src to dst
FILL	(addr count char –)	Fill count bytes at addr with char
HERE	(– addr)	Address of next free dictionary byte
ALLOT	(n –)	Reserve n bytes in dictionary
,	(n –)	Compile n as a cell into dictionary
C,	(char –)	Compile char as a byte into dictionary
CELLS	(n – n*cell-size)	Convert cell count to byte count
CELL+	(addr – addr')	Advance address by one cell
CHARS	(n – n)	Convert char count to byte count (=1 on byte-addressed)

Control Flow Words

Word	Stack Effect	Description
IF	(flag –)	Branch if flag is zero (skip to ELSE/THEN)
ELSE	(–)	Alternate branch after IF
THEN	(–)	End of IF...ELSE...THEN
DO	(limit start –)	Begin counted loop
LOOP	(–)	Increment index by 1; exit if index = limit

Word	Stack Effect	Description
+LOOP	(n -)	Increment index by n; exit when index crosses limit
LEAVE	(-)	Exit enclosing DO loop immediately
I	(- index)	Copy innermost loop index
J	(- index)	Copy outermost loop index
BEGIN	(-)	Mark start of BEGIN loop
UNTIL	(flag -)	Branch back to BEGIN if flag is zero
WHILE	(flag -)	Exit BEGIN loop if flag is zero
REPEAT	(-)	Branch back to BEGIN (used with WHILE)
AGAIN	(-)	Unconditional branch back to BEGIN
EXIT	(-)	Return from current word
EXECUTE	(xt -)	Execute word whose execution token is xt

Defining Words

Word	Stack Effect	Description
:	("name" -)	Begin word definition; enter compilation mode
;	(-)	End word definition; return to interpretation mode
IMMEDIATE	(-)	Mark most recently defined word as immediate
CONSTANT	(n "name" -)	Define a constant word that pushes n
VARIABLE	("name" -)	Define a variable; word pushes its address
CREATE	("name" -)	Create a named dictionary entry; no behavior
DOES>	(-)	Specify runtime behavior for CREATE'd words
FORGET	("name" -)	Remove named word and all newer words from dictionary
,	("name" - xt)	Push execution token of named word
[']	("name" - xt)	Compile-time version of tick (immediate)

I/O and String Words

Word	Stack Effect	Description
EMIT	(char -)	Output one character
KEY	(- char)	Wait for and return one keypress
CR	(-)	Output newline
SPACE	(-)	Output one space
SPACES	(n -)	Output n spaces
TYPE	(addr count -)	Output count characters from addr

Word	Stack Effect	Description
.	(n -)	Print top of stack in current base, followed by space
.S	(-)	Print entire stack contents without consuming
.R	(n width -)	Print n right-justified in field of width
U.	(u -)	Print top of stack as unsigned
U.R	(u width -)	Print unsigned right-justified in field
.(	("text" -)	Immediately print text up to closing paren
."	("text" -)	Compile string that prints at runtime
S"	("text" - addr count)	Compile string literal; push addr+count at runtime
ACCEPT	(addr maxcount - count)	Read line of input into buffer
WORD	(char - addr)	Parse next word from input, delimited by char
PARSE	(char - addr count)	Parse input to delimiter, return addr+count

Numeric Output Format Words

Word	Stack Effect	Description
HEX	(-)	Set numeric base to 16
DECIMAL	(-)	Set numeric base to 10
BINARY	(-)	Set numeric base to 2
BASE	(- addr)	Address of the base variable (default 10)
<#	(-)	Begin pictured numeric output
#	(ud - ud')	Convert one digit of pictured numeric output
#S	(ud - 0 0)	Convert all remaining digits
HOLD	(char -)	Insert char into pictured numeric output
#>	(xd - addr count)	End pictured numeric output; return string
SIGN	(n -)	Add minus sign to pictured output if n is negative

Block Words

Word	Stack Effect	Description
BLOCK	(lbn - addr)	Load block lbn into a buffer; return buffer address
BUFFER	(lbn - addr)	Assign buffer to lbn without reading; return address
LOAD	(lbn -)	Load and execute FORTH source in block lbn

Word	Stack Effect	Description
THRU	(lbn1 lbn2 –)	Load and execute blocks lbn1 through lbn2
FLUSH	(–)	Write all dirty buffers to storage
UPDATE	(–)	Mark current buffer as dirty (modified)
SAVE-BUFFERS	(–)	Synonym for FLUSH
EMPTY-BUFFERS	(–)	Invalidate all buffers without writing

Dictionary and System Words

Word	Stack Effect	Description
WORDS	(–)	Print all dictionary words
FIND	(addr len – xt flags 0)	Search dictionary by name
SEARCH-WORDLIST	(addr len wid – xt flags 0)	Search a specific wordlist
VOCABULARY	("name" –)	Define a new vocabulary (wordlist)
DEFINITIONS	(–)	Set current vocabulary as definition target
FORTH-WORDLIST	(– wid)	Push the id of the FORTH vocabulary
STATE	(– addr)	Address of compilation-state flag (0=interpret, -1=compile)
SEE	("name" –)	Decompile named word
ABORT	(–)	Reset stacks, print error message, restart interpreter
ABORT"	(flag "text" –)	If flag is non-zero, abort with message
QUIT	(–)	Return to outer interpreter (no error message)
BYE	(–)	Exit StarForth
INCLUDE	("filename" –)	Load and execute FORTH source file

Q48.16 Fixed-Point Words

Word	Stack Effect	Description
Q+	(q1 q2 – q1+q2)	Q _{48.16} addition
Q-	(q1 q2 – q1-q2)	Q _{48.16} subtraction
Q*	(q1 q2 – q1*q2)	Q _{48.16} multiplication
Q/	(q1 q2 – q1/q2)	Q _{48.16} division
INT>Q	(n – q)	Convert integer to Q _{48.16}
Q>INT	(q – n)	Convert Q _{48.16} to integer (truncate)
Q.	(q –)	Print Q _{48.16} value as decimal fraction

StarForth Physics Words

Word	Stack Effect	Description
WORD-ENTROPY	(-)	Print execution heat statistics for all words
PHYSICS-FREEZE	(-)	Suspend all adaptive runtime updates
PHYSICS-THAW	(-)	Resume adaptive runtime updates after freeze
PHYSICS-RESET-STATS	(-)	Clear all heat counters to zero

ACL Words (active only when ACL.4th is loaded)

Word	Stack Effect	Description
ACL-ENTRY	(xt - addr)	O(1) ACL table lookup by execution token
ACL-MODE@	(xt - mode)	Read enforcement mode of word
ACL-MODE!	(mode xt -)	Set enforcement mode (no-op if pinned)
ACL-PINNED?	(xt - flag)	Test whether word is pinned
ACL-PIN	(xt -)	One-way pin: freeze mode and decision permanently
ACL-STRICT	(xt -)	Set STRICT enforcement mode (no-op if pinned)
ACL-TTL-MODE	(xt -)	Set adaptive TTL mode (no-op if pinned)
ACL-TTL@	(xt - n)	Read current TTL counter
ACL-TTL!	(n xt -)	Write TTL counter
ACL-ALLOW@	(xt - flag)	Read cached access decision
ACL-ALLOW!	(flag xt -)	Write access decision
ACL-INHERIT	(src dst -)	Copy mode from src to dst; clear pin; reset TTL and decision
ACL-RECHECK	(xt -)	Recompute adaptive TTL and update decision
ACL-INIT-PRIMITIVES	(-)	Bulk-initialize ACL entries for all existing words
ACL-GRANT	(xt -)	Grant access to word
ACL-REVOKE	(xt -)	Revoke access to word

Appendix C

Glossary

ACL Access Control List. In StarForth, a per-word access-control mechanism implemented in `ACL.4th`.

Capsule An immutable, content-addressed VM initialization payload. A capsule's ID is its `XXHash64` content hash.

Cell The fundamental unit of the data stack. On a 64-bit host, one cell is 64 bits (8 bytes).

Colon definition A word defined with `:` and `;`.

Compilation mode The interpreter state entered after `:`, in which words are compiled rather than executed.

Data stack The primary working stack, distinct from the return stack.

Decay The process by which execution heat diminishes over time for quiescent words (`Loop #3`).

DoE Design of Experiments. StarForth's 2^7 factorial experimental framework for measuring the effect of each feedback loop.

Execution token (XT) An opaque value identifying a word in the dictionary, suitable for use with `EXECUTE` or `ACL` words.

Heat The execution-frequency metric tracked per word by `Loop #1`. Higher heat means more recent and frequent execution.

Immediate word A word that executes at compile time rather than being compiled into the current word definition.

Interpretation mode The default interpreter state in which words are executed as they are encountered.

Jacquard Selector The L8 steady-state machine (`ssm_jacquard.c`) that selects between run-time mode configurations: stable, volatile, diverse, temporal, transition, omni.

LBN Logical Block Number. The address scheme for StarForth's block storage subsystem.

LithosAnanke The bare-metal UEFI kernel that boots StarForth directly on hardware without a host operating system.

$\mathbb{Q}_{48.16}$ $\mathbb{Q}_{48.16}$ fixed-point format: 48-bit integer part, 16-bit fractional part. Precision of $1/65536 \approx 0.0000153$.

Return stack The secondary stack used for word-call linkage and temporary storage with `>R/R>`.

Rolling window The circular execution history buffer managed by **Loop #2**.

SMUDGE A flag set during word compilation that hides the partial definition from dictionary searches until **;** completes it.

SSM Steady-State Machine. The control structure governing StarForth's adaptive runtime.

Stack effect The notation (**before** - **after**) documenting what a word consumes and produces on the data stack.

Zuse The bootstrap superuser in StarForth's security model, named for Konrad Zuse.